Exam-Ready Study Guide

Distributed Systems · Systems Integration · Enterprise Systems Architecture · Cloud Computing · Data Warehousing & OLAP · Business Intelligence

Built only from your six uploaded chapter files (lecturer: Assoc. Prof. James Rwigema). Anything I add to help you understand, but which is **not on the slides**, is marked **[Added]**. Anything inconsistent or unclear in the slides is marked ⚠ **Flag**.

---

0. Source check: what I processed (read this first)

#	File	Slides read	Real title inside the file	Status
1	Chapter_1.pptx	59	Introduction to Distributed Systems	Text read in full; diagrams partly viewed
2	Chapter_2.ppt	58	Two parts: “Systems Integration” (slides 1–21) and “Distributed Systems Architecture for Enterprises” (slides 22–58)	Text read in full (converted from old .ppt format)
3	Chapter_3.ppt	37	Enterprise Systems Architecture	Text read in full (converted from .ppt)
4	Chapter_4_-_Cloud_computing.pptx	43	What is Cloud Computing?	Text, chart and speaker notes read in full
5	Chapter_5_-_Data_Warehouse.pptx	76	Data Warehousing, OLAP, OLTP, Data Marts	Text read in full; key diagrams viewed
6	Chapter_6_-_BI_ADS.pptx	65	Business Intelligence Overview (decision making to BI)	Text read in full; key diagrams viewed

All six files opened and were readable. **Problems in the source you should know about:**

1. ⚠ **Chapter 2 and Chapter 3 overlap.** Slides 22–58 of Chapter_2.ppt are word-for-word the same as all of Chapter_3.ppt (I compared them; the only difference is a typo in one figure caption). So the *Enterprise Systems Architecture* material appears twice. In this guide it is taught once, under **Chapter 3**, and Chapter 2 points to it.
2. ⚠ **Chapter numbering is inconsistent inside the files.** Chapter_2.ppt's first slide says "CHAPTER 1"; its middle slide says "CHAPTER 2"; Chapter_4's first slide says "CHAPTER 3"; figure numbers in Chapters 2/3 say "Figure 3-x". I follow your **file names** (Chapter 1 to 6).
3. ⚠ **Chapter 4 refers to "Chapter 3" for more on microservices and serverless.** That detail is not in any file you supplied, so I only give the one-line definitions from the slides.
4. ⚠ **Figures 3-1, 3-2, 3-4 and 3-6 (Chapters 2/3) all show the same picture** (a consumer and provider with request/reply arrows) after conversion. I cannot tell whether that is the lecturer's real figure or a file problem, so I describe those topics **from the slide text only**.
5. **Image-only slides I could not interpret with confidence** (no readable text): Chapter 1 slides 28, 44, 48, 52, 56; Chapter 6 slides 11, 22, 32, 34–36, 41–42, 44, 58 ("BI Types"). Where I describe these topics, it comes from nearby slide text.
6. Smaller internal inconsistencies are flagged in place with ⚠.

A **chapter-by-chapter coverage checklist** is at the very end.

PART I: COURSE OVERVIEW

1.1 The big picture

The course follows one story: **how organizations use many connected computers and shared data to run the business and make better decisions.**

- **Chapter 1** teaches the *technical foundation*: what a distributed system is, what goals it should meet and what makes it hard.
- **Chapter 2** moves to the *business problem*: organizations are split into departments ("silos") with separate systems, so they need **systems integration**, often through **ERP**.
- **Chapter 3** shows the *architecture* of ERP and enterprise systems: modules, tiers, web services, **SOA**, and a first look at the cloud.

- **Chapter 4** goes deep on **cloud computing**: the NIST model, deployment and service models, costs, migration, SLAs and **AWS**.
- **Chapter 5** covers where the data ends up: **data warehouses, OLAP, OLTP, data marts**.
- **Chapter 6** closes the loop: **decision making and Business Intelligence (BI)**, which uses the data warehouse, OLAP and data mining to support decisions.

```
Ch1 Distributed systems  -->  Ch2 Integrate siloed systems (ERP)
        |                             |
        v                             v
Ch4 Cloud computing  <--  Ch3 Enterprise architecture (tiers, SOA, cloud)
        |
        v
Ch5 Data warehouse / OLAP / OLTP  -->  Ch6 Business Intelligence & decisions
```

1.2 Foundational ideas to understand first

1. **Distributed system**: many independent computers that cooperate and look like one system to the user (Ch1).
2. **Transparency**: hiding the fact that things are spread out (Ch1).
3. **Client–server vs peer-to-peer**: the two ways of organizing roles (Ch1).
4. **Middleware**: the software layer that connects different computers and applications (Ch1, Ch2).
5. **Silo and integration**: separate departmental systems vs connected systems (Ch2).
6. **ERP and tiers**: one integrated software package, built in layers (Ch2, Ch3).
7. **Cloud: 5 characteristics, 4 deployment models, 3 service models** (Ch4).
8. **OLTP vs OLAP**: running the business (transactions) vs analysing the business (Ch5, Ch6).
9. **Data → Information → Knowledge → Decision → Action** (Ch6).

1.3 Realistic 60-minute plan

Minutes	Do this
0–3	Read Part I (this part)

Minutes	Do this
3–13	Chapter 1 (list-heavy: goals, challenges, hardware/software, architecture)
13–19	Chapter 2 (silos, 8 integration steps, ERP, benefits/limits)
19–26	Chapter 3 (modules, three-tier, SOA, cloud benefits/drawbacks)
26–35	Chapter 4 (NIST 5-4-3, models, costs, migration, SLA, AWS)
35–44	Chapter 5 (ETL terms, OLAP operations, ROLAP/MOLAP/HOLAP, OLTP vs OLAP)
44–51	Chapter 6 (8 decision steps, BI definition/components/layers)
51–56	Part III revision sheets (definitions, comparisons, memory aids)
56–60	Part V rapid revision and self-test. Use Part IV (mock exam) only if you finish early

If you run short of time: prioritise Chapter 4 (cloud) and Chapters 5/6 (OLAP, OLTP, BI): they are the most list- and definition-heavy and the easiest to score on.

PART II: COMPLETE CHAPTER-BY-CHAPTER NOTES

CHAPTER 1: Introduction to Distributed Systems

(Chapter_1.pptx, 59 slides)

1.A Chapter purpose

Teaches **what a distributed system (DS) is, why we build them, what we want from them (goals), what makes them hard (challenges), and how they are built (hardware, software, architecture models, types).**

1.B Key terminology

Term	Meaning (from the slides, in simple words)
Distributed system (DS)	A collection of independent computers that appear to the user as a **single coherent system**.
Distributed computing	One program split into parts that run **at the same time** on several networked computers.
Node	One computer/processing element in the system.
Middleware	Software layer **between the operating system and the applications**, connecting components across a network.
NOS	Network Operating System: each computer has its own OS with networking features.
DOS	Distributed Operating System: the OS on each computer knows about the others.
Multiprocessor	One computer with 2+ CPUs **sharing memory** and peripherals.
Multicomputer	Many computers, each with **its own memory**, communicating by messages.
SMP / ASMP	Symmetric / Asymmetric Multiprocessing.
NUMA	Non-Uniform Memory Access.
IDL	Interface Definition Language, used to specify interfaces (openness).
QoS	Quality of Service: guarantees about performance, security, reliability.

Term	Meaning (from the slides, in simple words)
RPC / RMI	Remote Procedure Call / Remote Method Invocation.
P2P	Peer-to-peer.
DoS attack	Denial-of-service attack (a security problem for DS).
MIPS	Millions of instructions per second (used in the speed example).

1.C Detailed teaching notes

1. What is a distributed system?

The slides give several definitions. Learn these:

1. **"A collection of independent entities that cooperate to solve a problem that cannot be individually solved."**
2. **"A collection of mostly autonomous processors communicating over a communication network."**
3. **"A system in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages."**
4. **"A collection of independent computers that appear to its users as a single coherent system."** (the most quoted one)

Distributed computing (the activity): one program is split into parts that run simultaneously on multiple computers communicating over a network.

Two aspects of a DS:

- **Hardware:** the computers are *autonomous* (independent).
- **Software:** users *think they are dealing with a single system*.

The slides also say **a parallel computer is implicitly a distributed system**, and give examples: a **network of workstations**, a **distributed manufacturing system (e.g., an automated assembly line)**, and a **network of branch-office computers**. Slide 3 gives a test for knowing you use one: a computer failing does **not** stop you from doing your work. ⚠ The slide wording of this sentence is garbled; learn the idea, not the exact words.

Case study: university registration

- *Situation:* A university has campuses in Kigali, Huye and Musanze. Students register for courses from any campus.
- *Concept applied:* a distributed system.
- *How:* (1) Each campus runs its own computer (autonomous hardware). (2) The computers talk by sending messages over the network. (3) The student sees one registration system (single coherent system). (4) If the Huye server crashes, registration still works from the others.
- *Lesson:* many independent machines + messages + one-system illusion = a DS.

Exam angle: "Define a distributed system" → give the single-coherent-system definition **and** mention independent computers, a network, and message passing.

2. Ingredients of a DS (layered view)

Each **host** (computer) has the same layers, bottom to top:

```
+-------------------------------------+
| Component-1 ... Component-n         |  <- application components
+-------------------------------------+
| Middleware                          |  <- connects components across hosts
+-------------------------------------+
| Network OS                          |
+-------------------------------------+
| Hardware                            |
+-------------------------------------+
   Host-1 ... Host-n  all joined by a NETWORK
```

One slide shows hosts **without** middleware (only Network OS); the next shows hosts **with** middleware joined by a network. The point: **middleware is what turns a set of networked computers into a distributed system.**

3. Why parallel and distributed computing?

The slides give only fragments here: the 1990s trend was **instruction-level parallelism (ILP)**, and multi-core parallel systems are going to **outperform** mainstream ones. They list open difficulties:

- how to use a large number of processors at the same time;
- how to write parallel programs easily and efficiently;

- what software support is needed;
- analysing and optimising parallel programs is still hard;
- how to debug concurrent programs.

Slide 3 also gives the real-world motivation: the need for **more information usage and access regardless of where it is located**, and the growth of the Internet and the "global village".

4. Advantages of a DS

A. Over centralized systems (a single big mainframe):

Advantage	Meaning	Slide example
Economics	Many microprocessors give better **price/performance** than a mainframe	
Speed	More total computing power than a mainframe; load distribution	10,000 CPU chips at 50 MIPS each
Inherent distribution	Some applications are naturally spread out	A supermarket chain
Reliability	If one machine crashes the whole system survives; higher availability	
Incremental growth	Add power in small steps (modular expandability)	

Another driving force: the huge number of PCs and the need to **collaborate and share information**.

B. Over independent PCs:

Advantage	Meaning
Data sharing	Many users access a common distributed database
Resource sharing	Share expensive peripherals (e.g., colour printers)
Communication	Better human-to-human communication (email, chat)
Flexibility	Spread the workload over available machines

Memory aid for A: E-S-I-R-I: *Economics, Speed, Inherent distribution, Reliability, Incremental growth.*

5. Disadvantages of a DS

1. **Software:** hard to develop software for DS.
2. **Network: saturation** (overloaded) and **lossy transmissions** (messages lost).
3. **Security:** easy access also applies to **secret data**.

Exam angle: advantages/disadvantages questions want **3–5 named points, each with a one-line explanation**. Use the tables above.

6. Goals of a DS

The main goal is to **let users access and share resources**, while meeting three goals: **Transparency, Openness, Scalability.**

Goal 1: Transparency

What it is: hiding the fact that processes and resources are **physically distributed** across multiple computers.

The 8 types of transparency (slide table):

Type	What it hides
Access	Differences in **data representation** and in how things are invoked
Location	**Where** an object resides

Type	What it hides
Migration	That an object can **change its location** (hidden from the object)
Relocation	That the system can move an object **while a client is using it** (hidden from the client)
Replication	That an object has **copies** in different places
Concurrency	The **coordination** of activities between objects/users sharing resources
Failure	That something **failed** and was **recovered**
Persistence	That an object may be (partly) **stored on disk or in memory** ("passivated") by the system

Memory aid: A-L-M-R-R-C-F-P: *All Lazy Men Really Relax, Chatting For Pleasure* (Access, Location, Migration, Relocation, Replication, Concurrency, Failure, Persistence).

Easy confusion: Migration vs Relocation. Migration: the object moves and *it* does not notice. Relocation: the object moves while a *client is using it* and the *client* does not notice.

Limits of full transparency (the lecturer's three arguments):

1. Users may be on different continents; the distribution is **obvious** and not something you want to hide.
2. **Completely hiding failures is impossible**, in theory and practice: you cannot distinguish a **slow** computer from a **failing** one, and you can never be sure a server finished an operation before it crashed.
3. Full transparency **costs performance**, e.g., keeping Web caches exactly up to date with the master copy, or immediately flushing writes to disk for fault tolerance.

Exam angle: "Explain transparency and discuss its limits" → definition, the 8 types with one-liners, then the 3 limits.

Goal 2: Openness

What it is: offering services according to **standard rules that describe the syntax and meaning (semantics)** of the services, e.g., interfaces written in an **IDL**.

An **open distributed system** can interact with services from other open systems regardless of the underlying environment. It should:

- conform to **well-defined interfaces**;
- support **portability** of applications;
- **interoperate** easily.

Challenge: make the DS independent of the **heterogeneity** (differences) of **hardware, platforms and languages**.

Implementing openness: the system must support different **policies** chosen by applications and users, for example: What consistency do we need for cached data? What may downloaded code do? Which QoS do we adjust when bandwidth varies? How secret must communication be?

Ideally the DS provides only **mechanisms**, not fixed policies: allow dynamic caching policies (per item), different trust levels for mobile code, adjustable QoS per data stream, and different encryption algorithms.

In simple words: a policy is *what you decide*; a mechanism is *the tool that lets you decide*. An open system gives tools and lets users set the rules.

Goal 3: Scalability

The slides warn that people say “scalable” without explaining why. Three dimensions must be considered:

Dimension	Meaning
Size scalability	Number of **users and/or processes**
Geographical scalability	Maximum **distance between nodes**
Administrative scalability	Number of **administrative domains** (organizations)

Most systems handle only size scalability, and the “(non)solution” is **powerful servers**. Today the challenge is **geographical and administrative scalability.**

Techniques of scaling (4):

Technique	Idea	Examples from slides
Hiding communication latency	Avoid waiting for remote replies	Move computation to clients
Distribution	**Partition** data and computation across machines	Java applets; **DNS** (decentralized naming); the **WWW**
Replication	Make **copies** of data on different machines	Replicated file servers/databases, **mirrored Web sites**, distributed shared memory
Caching	Let clients use **local copies**	Web caches (browser/proxy), file caching

The scaling problem: applying these techniques is easy **except** that multiple copies cause **inconsistency**: changing one copy makes it differ from the rest. Keeping all copies consistent needs **global synchronization** on every change, which makes large-scale solutions practically impossible. If an application can **tolerate some inconsistency**, global synchronization can be reduced, and what is tolerable **depends on the application**.

Example: two lecturers' phones cache the same timetable. One changes it. Until the other refreshes, they see different timetables (inconsistency). Demanding that every phone updates instantly would slow the whole system.

7. Challenges of a DS (8)

#	Challenge	Meaning (slides)
1	**Heterogeneity**	Built from different networks, operating systems, computer hardware and programming languages
2	**Openness**	DS should be **extensible**; integrating components written by different programmers is hard
3	**Security**	**Encryption** protects shared resources and keeps data secret in transit; **denial-of-service (DoS) attacks** are still a problem

#	Challenge	Meaning (slides)
4	**Scalability**	A DS is scalable if the **cost of adding a user is a constant amount** of resources
5	**Failure handling**	Any process, computer or network may fail **independently** of the others; design must deal with each failure
6	**Concurrency**	Many users send **concurrent requests**; each resource must be **safe** in a concurrent environment
7	**Transparency**	Make aspects of distribution **invisible to the application programmer** so they only design their application
8	**Quality of Service (QoS)**	Not enough to give access; must also **guarantee** performance, security, reliability

Memory aid: H-O-S-S-F-C-T-Q → *Heterogeneity, Openness, Security, Scalability, Failure, Concurrency, Transparency, QoS.*

⚠ **Flag:** *Openness, Scalability and Transparency appear as both goals (slide 14) and challenges (slides 22–23).* Treat them as goals you want, and as things that are hard to achieve.

8. False assumptions when developing a DS (the 8 fallacies)

Developers wrongly assume that:

1. The network is **reliable**
2. The network is **secure**
3. The network is **homogeneous**
4. The **topology does not change**
5. **Latency is zero**
6. **Bandwidth is infinite**
7. **Transport cost is zero**
8. There is **one administrator**

Every one is **false**, and each causes bugs. *(Latency = delay for a message to travel.)*

9. Hardware concepts

Three categories: **Multiprocessors, Multicomputers, Networks of computers.**

Multiprocessor: a computer system with **two or more processing units sharing main memory and peripherals**, so programs are processed simultaneously. Processor symmetry: **SMP** (all CPUs treated equally), **ASMP** (some CPUs reserved for special purposes). Other types listed: **NUMA** and **clustered multiprocessing**.

Communication architecture (two ways):

Message passing	Shared memory
Separate address space for each processor	**Shared address space**
Communicate by **messages**	Communicate by memory **read/write**
Processors have local message-queue memories	Easy on small machines
Focus on costly non-local operations	**Lower latency**

Distinguishing features: private vs shared memory and **bus vs switched** interconnection.

Networks of computers (slide 29): very high **node heterogeneity** (parallel systems, high-end PCs/servers, simple network computers, mobile computers) **and network heterogeneity** (local gigabit LANs, wireless, long-haul high-latency POTS connections, wide-area switched megabit links). **Ideally the DS hides these differences.**

10. Software concepts: DOS vs NOS vs Middleware

Distributed Operating System (DOS). The OS on each computer **knows about the other computers**, and services are **transparently distributed**. For **multicomputer** OS: harder than a normal multiprocessor OS because **memory is not shared**, so everything depends on **message passing**. Difficulties: often **no simple global communication** (only bus-based multicomputers have hardware broadcast); **no simple system-wide synchronization**; **virtual (distributed) shared memory** needs the OS to keep a global memory map in software; **resource management has no central point**. In practice **very few truly multicomputer OS exist** (example: **Amoeba**).

Network Operating System (NOS). Each computer has **its own OS** with networking facilities. Computers work **independently** (may even run **different OSs**). **Services are tied to individual nodes** (ftp, telnet, WWW). **Highly file-oriented**: processors basically

share **files only**.

Middleware. The software that **connects software components or enterprise applications**; a layer **between OS and applications**. The OS on each computer **need not know** about the others and **need not be the same**; services are generally **transparently distributed**.

Why middleware was needed: too many networked applications were hard to integrate (different departments running different NOSs; integration only at primitive NOS service level). Needs: **federated information systems** (combine different databases but give one view), **enterprise-wide Internet services** using existing information systems, **transactions across databases**, **extensibility** for future services (mobility, tele-working). **Constraint:** use the **existing operating systems** as the underlying environment.

Middleware services (4 groups):

Group	Examples (slides)
Communication	Procedure calls across networks, remote-object method invocation, **message-queuing** systems, advanced streams, **event notification** (replaces primitive socket message passing)
Information system	System-wide **naming** services, advanced **directory** services (search engines), **location** services for mobile objects, **persistent storage**, data **caching and replication**
Control	**Distributed transaction processing**, **code migration**
Security	**Authentication and authorization**, simple **encryption**, **auditing**

Comparison NOS / DOS / Middleware. The slide lists the criteria only: degree of transparency; same OS on each node?; number of copies of the OS; basis for communication; how resources are managed; easy to scale?; how open? *The values are not on the slide text.* **[Added]** A standard answer you can use (consistent with the characteristics given above):

Criterion	DOS	NOS	Middleware
Degree of transparency	Very high	Low	High
Same OS on each node?	Yes	No	No
Basis for communication	Messages / shared memory	**Shared files**	Model-specific (e.g., remote calls, messages)
Resource management	Global, distributed	**Per node**, by the node's own OS	Per node

11. Architecture models

The **architecture** of a system is its **structure in terms of separately specified components and their interrelationships.** The goal: the structure meets **present and likely future demands** (like the architecture of a building).

Four key questions (architectural elements):

1. What are the **entities** that communicate?
2. **How** do they communicate (**communication paradigm**)?
3. What **roles and responsibilities** do they have?
4. How are they **mapped onto the physical infrastructure (placement)**?

1. Entities. Normally **processes**; in sensor networks, **nodes**; often **threads** are the real endpoints. Problem-oriented abstractions: **objects** (encourage OO approaches) and **components** (like objects but accessed through interfaces).

2. Communication paradigms (3):

- **Inter-process communication:** message-passing primitives and direct use of Internet-protocol APIs (**socket programming**).
- **Remote invocation:** a two-way exchange that calls a remote operation. Includes **request-reply protocols** (support client–server computing), **RPC** (call procedures in remote processes **as if local**), and **RMI** (like RPC but for **distributed objects**). In all of these, **sender and receiver are directly linked (two-way relationship).**

- **Indirect communication:** senders **do not need to know who receives** (space uncoupling) and sender and receiver **need not exist at the same time** (time uncoupling). ⚠ The slide says “time coupling”, which contradicts the meaning; the correct idea is **time uncoupling**.

3. Roles and responsibilities:

- **Client–server:** clients ask server processes to access shared resources the servers manage. **Servers may themselves be clients of other servers.** Example: a **Web server**. A service may also be provided by **multiple servers**.
- **Peer-to-peer (P2P):** all processes play **similar roles**, cooperating as peers **without client/server distinction**.

4. Placement: how entities map to the physical infrastructure; **depends on application knowledge; few universal guidelines.** Strategies: **mapping a service to multiple servers**, **caching**, **mobile code**, **mobile agents**.

- **Caching:** a **cache** is a store of recently used data objects **closer to the client** than the objects themselves. New objects replace old ones; on a request the cache service first checks for an **up-to-date copy**, otherwise fetches one. Caches can sit **with each client** or in a **shared proxy server**.
- **Mobile code:** code downloaded to run locally when accessing a service (e.g., a website needing extra code).
- **Mobile agent:** a **running program (code and data)** that **travels from computer to computer** doing a task on someone’s behalf (e.g., collecting information) and **returns with results**.

12. Types of DS (by node arrangement)

Client/Server systems and **Peer-to-peer systems.**

Client–server model:

- **Servers** provide services around a shared resource: file systems, databases, repositories; shared linked documents (Web, VoD); shared applications; shared distributed objects.
- **Clients** give remote access to services: a programming interface that turns local service calls into **request/reply messages**; simple devices (barcode readers, teller machines, phones); computers with their own user interface for a service; computers with an integrated UI for related services (compound documents).
- A slide titled **Application layering** exists (diagram not readable).

P2P model: a distributed application architecture that **partitions tasks or workloads between peers**. Peers are **equally privileged, equipotent participants**; they form a network of nodes and **make part of their resources (processing power, disk storage, bandwidth) directly available to others without central coordination by servers or stable hosts.**

1.D Important comparisons (Chapter 1)

	Client–Server	Peer-to-Peer
Roles	Distinct: client asks, server provides	**Equal** peers, all similar roles
Control	Servers manage shared resources (central)	**No central coordination**
Example	Web server	File sharing among peers **[Added]**

	Multiprocessor	Multicomputer
Memory	**Shared**	**Private** (separate address spaces)
Communication	Memory read/write	**Message passing**

	Transparency	Openness
Question answered	"Can users **ignore** the distribution?"	"Can **other systems/vendors** plug in via standard interfaces?"

1.E Exam-ready model answers (Chapter 1)

Q1. Define a distributed system and give two advantages over a centralized system. A DS is a collection of independent computers that appear to users as a single coherent system, communicating by passing messages. Advantages: (1) **Economics:** microprocessors give better price/performance than a mainframe. (2) **Reliability:** if one machine crashes the rest keep working, so availability is higher. *(Also: speed, inherent distribution, incremental growth.)*

Q2. Explain the three goals of a DS. Transparency: hide that resources are distributed (access, location, migration, relocation, replication, concurrency, failure, persistence). **Openness:** services follow standard interfaces (IDL) so different systems interoperate and applications are portable. **Scalability:** the system keeps working as users (size), distance (geographic) and organizations (administrative) grow; techniques are hiding latency, distribution, replication and caching, whose problem is **inconsistency**.

Q3. Distinguish NOS, DOS and middleware. NOS: each computer has its own OS, nodes work independently and share mainly **files**. DOS: OS on each node knows the others and distributes services transparently (few exist, e.g., Amoeba). Middleware: layer **above** the local OSs (which may differ) that provides communication, information, control and security services.

Q4. Why can full transparency not be achieved? Distribution is sometimes obvious (different continents); failures cannot be completely hidden because a slow computer cannot be told from a failed one; and transparency costs performance (e.g., always-fresh caches).

1.F Chapter 1 recap

- DS = independent computers + network + messages + **single-system illusion**.
- Advantages: **ESIRI** (over centralized) and data/resource sharing, communication, flexibility (over PCs). Disadvantages: **software, network, security**.
- Goals: **Transparency (8 types), Openness, Scalability (3 dimensions, 4 techniques, consistency problem).**
- 8 challenges and 8 false assumptions.
- Hardware: multiprocessor (shared memory) vs multicomputer (messages). Software: DOS, NOS, **middleware**.
- Architecture: entities, communication paradigm, roles (client–server, P2P), placement (caching, mobile code, mobile agent).

CHAPTER 2: Systems Integration (and silos)

(Chapter_2.ppt: slides 1–21 are the "Systems Integration" part; slides 22–58 repeat Chapter 3, see Chapter 3 notes.)

2.A Chapter purpose

Explains **why organizations broke into departments (silos), why separate information systems became a problem, and how systems integration (logical and physical), often via ERP, solves it** — plus the benefits, limits, and management and ethical implications.

2.B Key terminology

Term	Meaning
Silo	A **compartmentalized operating unit isolated from its environment**.
Horizontal silo	Division by **function/department** (Accounting, HR).
Vertical silo	Division by **hierarchical level** (strategic, tactical, operational).
POSDCORB	Luther Gulick's functions: **P**lanning, **O**rganizing, **S**taffing, **D**irecting, **Co**ordinating, **R**eporting, **B**udgeting.
BPR	Business Process Re-engineering: redesigning work as **cross-functional business processes**.
Cross-functional process	A process using people/resources from **several departments** that share information at any level.
Systems integration	Making separate information systems share data and connect (**logically and physically**).
ERP	**Enterprise Resource Planning**: integrated, multi-module application packages serving several business functions.
Legacy system	Older existing applications.
Single sign-on	One login giving access to all integrated systems.
EDI / B2B	Electronic Data Interchange / Business-to-Business commerce.
ROI	Return on investment.

2.C Detailed teaching notes

1. Functional silos

What it is: organizations break complex work into smaller manageable tasks assigned to groups who can be **held responsible**.

- **Horizontal silos:** based on **POSDCORB** (Luther Gulick, from the late 1930s), which produced formal functions such as control, management, supervision and administration. Departments such as **Accounting** and **Human Resources** reflect this. *(Slide figure: Functional model of organization.)*
- **Vertical silos:** roles split into **layers**: **CEOs/Presidents** plan **long-term strategy**; **middle management** handles **tactical** issues and carries out policy; **lower management** runs **day-to-day operations**. *(Figure: Hierarchical model.)*

Why it exists: as organizations grow they break functions into smaller units so staff can **manage complexity and specialize**, which boosts productivity and efficiency.

Example: a hospital has separate Pharmacy, Billing and Wards departments, each with its own system. Billing cannot see what Pharmacy dispensed, so invoices are wrong. That is the silo problem.

2. Business process and silos

- The silo problem **gave birth to business process re-engineering (BPR).**
- A **cross-functional business process** involves people and resources from various departments **working together and sharing information at any level.**
- A **cross-functional (matrix) structure breaks the silos by opening information flows between departments.** *(Figure 2-3: Matrix structure of organization.)*

3. Evolution of information systems (IS)

- Functions (sales, manufacturing, HR) give the organization a structure that lets it run smoothly.
- **A silo information system is inefficient, inaccurate and expensive**; it creates **bottlenecks**, and information is **not available in real time.**
- The role of IS has been to **support the organization's evolving information needs.**

4. IS architectures and functionalization

- Rapid advances in computer and network technology and changing organizational dynamics create new IS models.
- **Web-based systems use a distributed architecture** that shares applications and data between client and server computers. PCs connect through a network to a **Web server** (a window to an application) and a **database server** (a mainframe or other computer).
- **IS functionalization:** besides serving the **management levels**, IS supports the **major functions** (manufacturing, marketing, accounting, finance, HR). Each functional area has **different information needs and reports**, and **several levels of management**, each needing **different analysis and detail**. *(Figure 2-6: IS categorized by functional and hierarchical models.)*

5. Systems integration: logical and physical

	Logical integration	Physical integration
Idea	Build IS that let the organization **share data with all stakeholders based on need and authorization**	Provide **seamless connectivity between heterogeneous systems**
Needs	Management **changes organizational structure, processes, and employee roles/responsibilities**	**BPR** changes employees' mindset and enables them to do tasks **a new way**

In simple words: logical = *change how the organization works*; physical = *connect the technology*.

6. The 8 steps in integrating systems

Step	Name	What it means
1	**Resource categorization**	Provide **IT support** for an integrated environment to avoid support and maintenance problems
2	**Compliance and standards**	A **single sign-on policy** so employees and partners can access the system anytime, anywhere

Step	Name	What it means
3	**Legacy systems support**	A **policy for supporting older legacy applications**
4	**Middleware tools**	Essential **in the short term** if existing applications must be kept
5	**Authentication and authorization policies**	Single sign-on policy for application and data access
6	**Centralized IT services and support**	A **centralized IT help desk** supporting all applications and platforms
7	**Back-up, recovery and security**	A good back-up/recovery system for failure or **major disaster**
8	**Hardware and software standardization**	Standards and policy for **acquiring** new hardware/software, aligned with IT strategy

Memory aid: *"Really Careful Lawyers Make Absolutely Correct Backups & Standards"* → Resource categorization, Compliance, Legacy, Middleware, Authentication, Centralized services, Back-up, Standardization.

⚠ **Flag:** steps 2 and 5 are described with almost the same single sign-on sentence on the slides. Learn them as: **step 2 = compliance/standards (the policy)**, **step 5 = authentication/authorization (who may access what)**.

7. Benefits and limitations of systems integration

Benefits	Limitations
Increased revenue and growth	**High initial set-up costs**
Leveling the competitive environment	**Power and interdepartmental conflicts** (because information is shared)
Enhanced information visibility	**Long-term, intangible ROI** (usually several years)
Increased standardization	**Creativity limitations** (restricts creativity and independence)

8. ERP and systems integration

ERP: integrated, multi-module application software packages designed to serve and support **several business functions** across an organization. Usually **commercial packages** that collect and integrate information from many areas. They let the organization **standardize and improve processes and implement industry best practices.**

- **ERP and logical integration:** ERP makes organizations focus on **business processes rather than functions**; it has **built-in processes** for common functions. Example (slides): processing a **customer order** through **order entry → routing through departments → communicating output to parties**.
- **ERP and physical integration:** before installing ERP an organization may need to **upgrade or install middleware** or **remove legacy hardware/software**. Integration is needed at the **data level, client level and application level.** A good ERP improves efficiency with processes aimed at **organizational rather than departmental goals**, a **paperless flow**, and **EDI / B2B** commerce with partners.

9. Implications for management

- **Silos do not work:** organizations lose in the long term when information is **not shared in real time** across functional boundaries.
- **Hidden benefits:** decision-making can be **cascaded to all departments**; **lower-level employees** make better decisions when dealing with clients or partners.
- **Challenges:** replacing old hardware and software; working with IT consultants; **human challenges** (impact on IT staff, department heads losing control of data, rumors of layoffs).
- **Ethical issues:** employees may **exploit information for personal advantage** or access it illegally. **Remedies:** policies on **ethical use of information**, **security software and hardware (e.g., firewalls)**, and **training and education** on accessing information.

2.D Case study: integrating a university

- *Situation:* Finance, Admissions and the Library each have separate systems; a student's fee payment is not visible to the Library, so a paid-up student is blocked.
- *Concepts:* silos → **logical integration** (one process: "enrol student" across departments, shared data by authorisation) and **physical integration** (**middleware** linking old Library software to new Finance software, **single sign-on**).

- *Steps:* categorise IT support (1) → set standards/single sign-on (2) → support the legacy Library system (3) → use middleware (4) → define authentication (5) → central help desk (6) → back-ups (7) → standardise purchasing (8).
- *Lesson:* integration needs both **technology and organizational change**; expect **cost, conflict and a long payback**.

2.E Important comparisons (Chapter 2)

	Horizontal silo	Vertical silo
Divided by	**Function** (Accounting, HR…)	**Management level** (strategic / tactical / operational)

	Logical integration	Physical integration
Focus	**Data sharing + organizational/process change**	**Technical connectivity** between heterogeneous systems

2.F Exam-ready model answers (Chapter 2)

Q1. What is a silo and why is it a problem? A silo is a compartmentalized operating unit isolated from its environment. Silo information systems are **inefficient, inaccurate and expensive**, create **bottlenecks** and do not give **real-time** information across departments. BPR and integration were created to fix this.

Q2. Distinguish logical from physical integration. *(Use the table in 2.C-5.)*

Q3. List the steps in integrating systems. The 8 steps: resource categorization; compliance and standards; legacy systems support; middleware tools; authentication and authorization policies; centralized IT services/support; back-up, recovery and security; hardware/software standardization.

Q4. Benefits and limitations of integration. Benefits: revenue/growth, level competition, visibility, standardization. Limitations: set-up cost, power conflicts, long intangible ROI, reduced creativity.

2.G Chapter 2 recap

- Silo = isolated unit; **horizontal (function/POSDCORB)** and **vertical (levels)**. Silo systems = slow, inaccurate, costly.
- **BPR → cross-functional processes → integration (logical + physical).**
- **8 steps**; **4 benefits / 4 limitations**; **ERP** = integrated multi-module package implementing best practices.
- Management: silos fail; challenges include people issues; **ethics** needs policy, security and training.

CHAPTER 3: Enterprise Systems Architecture (ESA)

(Chapter_3.ppt, 37 slides. Identical to slides 22–58 of Chapter_2.ppt.)

3.A Chapter purpose

Shows **how enterprise systems (mainly ERP) are structured**: functional modules, layered/tiered architecture, web services, **SOA**, and the cloud, with benefits and drawbacks of each.

3.B Key terminology

Term	Meaning
ESA	Enterprise Systems Architecture: has a **functional** side (ERP modules) and a **system** side (hardware, software, networking).
ERP module	Software component supporting one business function (accounting, HR…).
Business rules	Rules through which ERP embeds the organization's policy and procedure.
Tier / layer	A level of the architecture that separates responsibilities.

Term	Meaning
Web tier / Application tier / Data tier	Presentation access / business processes and reporting / organizational data.
Server-centric / client-centric	Most processing on the server / on the client's computer.
SOA	Service-Oriented Architecture.
Web service	Interface letting different applications work together by **sending and receiving messages**.
WSDL / UDDI / SOAP	Web Services Description Language / Universal Description, Discovery and Integration / Simple Object Access Protocol.
XML	The language Web-service protocols are based on.
ECM	Enterprise Content Management.
OLAP	Online Analytical Processing (see Chapter 5).
SSL / ASL / JVM	Secure Socket Layer / Applications Software Layer / Java Virtual Machine (Info.Net example).
SOX Act	Sarbanes-Oxley Act (financial reporting compliance).
RFID	Radio-frequency identification (data collection).

3.C Detailed teaching notes

1. Components of ESA

- **Functional:** the **ERP modules** supporting business functions (**Accounting, Human Resources, Procurement, Fulfillment**, etc.).
- **System:** the ERP **architecture** viewed through **hardware, software and networking** components.

2. ERP modules

The key role of ERP is to support **accounting, sales, inventory control and production**. Vendors **SAP, Oracle and Microsoft** provide modules for the major functional areas. ERP software **embeds best business practices** through **business rules**.

Modules from three vendors (slide table):

Function	SAP	Oracle / PeopleSoft	Microsoft Dynamics
Sales	Sales and Distribution, Sales Opportunity	Marketing and Sales, Supply Chain Mgmt	Retail POS, Field Service Mgmt
Procurement	Purchasing, Supplier Relationship Mgmt	Procurement and Supplier Relationship Mgmt	Supply Chain Mgmt
Production	MRP, Product Life Cycle Mgmt	Manufacturing	Manufacturing
Accounting	Financial Accounting	Financial Management	Financial Management
Distribution	Warehouse Management	Supply Chain Management	Distribution Management
Customer service	CRM	CRM	CRM
Corporate performance and governance	Governance, Risk and Compliance Mgmt	Corporate Performance Mgmt	Analytics
Human resources	Human Capital Management	Human Capital Management	HR Management
Miscellaneous	Banking	Campus Solutions	E-commerce, Portals

What each module does:

- **Production:** plans and optimizes **manufacturing capacity, parts, components and materials** using **historical production data and sales forecasts**.
- **Purchasing:** streamlines **procurement of raw materials and supplies**.

- **Inventory management:** keeps the **right stock level** in the warehouse.
- **Sales and marketing: order placement, order scheduling, shipping, invoicing**.
- **Finance:** gathers financial data from departments and produces **financial reports**.
- **Human resource:** streamlines management of **people (human capital)**.
- **Miscellaneous (non-traditional): business intelligence, self-service, project management, e-commerce**.

Benefits of key modules:

Module	Benefits (slides)
Self-services	Flexible support for employees; simplified access to information
Performance management	**Real-time, personalized measurements and metrics**; executives get business statistics and **key performance measures**
Financials	Compliance and predictability; deeper financial insight and control; **automated accounting**; rigorous financial reporting support (**SOX Act**)
HR management	Attract, develop and **retain** talent; align efforts with corporate goals; **standardized, automated** processes aid compliance with regulations; build **project teams by skills/availability**; track time and results; analyze **workforce trends and planning**
Procurement and logistics	Savings by automating routine tasks (e.g., **requisitions → purchase orders**, **electronic catalogs**); **supplier integration** and collaboration; better resource use (**cross-docking, RFID, bar codes**); more productive goods movement; lower **transportation costs**
Product development and production	**Shorter time to market**; higher quality and timely delivery; **real-time visibility** (availability check)

Module	Benefits (slides)
Sales and service	More orders processed, lower admin cost; accurate **customer information**; **mobile access** for field staff; less travel cost; **EHS** reporting compliance; better **incentive and commission** management; real-estate management

3. ERP architecture and three-tier architecture

ERP architecture is organized in **layers (tiers)** to **manage complexity and provide scalability and flexibility.** The **three-layer architecture** is called **the most reliable, flexible and scalable** and the most common today: **Web servers, Application servers, Database servers.**

```
  [ Users / Web browser ]
          |
  +-------------------+
  |   WEB TIER        |  web portal: users access and analyse info through a
browser
  +-------------------+
  |   APPLICATION TIER|  business processes + reporting; shields users from
ERP internals
  +-------------------+
  |   DATA TIER       |  structure of all organizational data and
relationships
  +-------------------+
```

Benefits of three-tier: Scalability (easy to add/change/remove applications), **Reliability** (multiple levels of redundancy), **Flexibility** (partitioning is simple), **Maintainability** (lower support cost), **Reusability** (reusable components), **Security** (IT staff have more control). **Limitation:** can be **very expensive and complex.**

Memory aid: SR-FM-RS: *Scalability, Reliability, Flexibility, Maintainability, Reusability, Security.*

Layered example: Info.Net. The layered ERP architecture generalizes functional layers so it can change with new technology. Steps: user has a **Java-enabled browser** → PC connected to **Intranet and Internet** to reach an Info.Net server → user interacts with the **Java Virtual Machine interface layer** to set up a **secure SSL connection** → then talks to the server through the **applications software layer (ASL)**.

Infrastructure requirements: traditional networks need **upgrading before ERP** and must be in the budget; a **high-availability network** is required, which grows with users and supply-chain integration; integration with **partner and customer systems** lets a company manage **order tracking and inventory**; **OLAP** gives the ability to access, present and analyse data **across dimensions.**

4. Web services architecture

Web architecture is often described as a **fourth tier**: the Web tier is split into a **Web services tier** and a **Web browser tier.** Internet access technologies: **Web server, ERP portal, back-end server integration, browser plug-ins or applets.** *(Figure: PeopleSoft's server-centric Internet architecture.)*

	Benefits	Drawbacks
Web-based (general)	Many users reach ERP over the Web; easy to integrate with existing systems; better system-to-system integration	
Server-centric	**No complex, expensive client software**; **secure** user access	**Slower**
Client-centric	**Better response time** (processing on the client's computer)	**Lacks security**

5. Service-Oriented Architecture (SOA)

What it is: also called **object-oriented architectures for Web platforms**; it **breaks the business tier into smaller, distinct services** that together support an ERP functional module. Any **service consumer** can exchange messages with any **service provider**, from any device, OS or language.

SOA is a software development model based on a contract between consumer (client) and provider (server) that specifies:

1. **Functional description** of the service;
2. **Input requirements and output specifications**;
3. **Precondition** (environment state before the service can be invoked);
4. **Postcondition** (environment state after the service has run);
5. **Error handling** when something breaks down.

Benefits (technical): services across **heterogeneous platforms**; **location independence** of business logic; services can exist **anywhere**; **loose coupling**; **granular authentication/authorization**; dynamic search and connectivity; **better reliability**; **lower hardware cost**; faster standards-based consolidation; a **data bridge between incompatible technologies**; **composite applications**; a **self-healing infrastructure**; **real-time decision-making applications**; a **unified taxonomy of information**.

Business-value benefits: meet **customer demand faster**; **lower acquisition/maintenance cost**; management of business functions **closer to business units**; **leverage existing investments**; **less reliance on expensive custom development**.

Drawbacks: costly and time-consuming; needs **complex firewalls** for service communication; **inconsistent performance**; needs **enterprise-level focus**; **sophisticated security**; **high cost because services need to be "junked" (discarded) often** *(slide wording, ⚠ unclear)*.

SOA and Web services: Web services are **interfaces letting different applications and components work together**; they interact **only by sending and receiving messages**; built on **open standards: WSDL, UDDI, SOAP**; the protocols are **XML-based**.

SOA and Enterprise Content Management (ECM): ECM is enterprise software that **stores, preserves, manages and delivers content tied to business processes**; it supports **business goals, not just content**; vendors now see it as **no longer application-specific.**

6. Cloud architecture (as presented in Chapter 3; full detail in Chapter 4)

- **Cloud computing:** a **software service provided over the Internet, securely, by a provider on a monthly or yearly lease.**
- Companies **save money** by replacing **per-seat licensed software**; some providers also let you **build and host your own applications** on their engines.
- Good for organizations that do **not** want to **buy/install/maintain software** or worry about **security, privacy and legal issues** of data storage. **Risk:** reliance on **external vendors** for reliability, security and continuity.

Benefits	Drawbacks
Pay a **subscription**, not licenses/upgrades	**Data security**
Lower **capital and operating** expenses	**Vulnerability**

Benefits	Drawbacks
Access from **anywhere** with Internet	**Conflict of interest** (provider could copy your application)
Nothing to install	**Not suited to highly competitive industries** (e.g., biotech) where **intellectual property** is hard to protect
Dynamic scalability on demand	
No maintenance fees	
Green computing (slide claim: clean energy)	
Guaranteed reliability (slide claim)	

7. Implications for management

- Enterprise architecture is important for the **long-term functioning** of the organization.
- ERP architecture decisions are **complex**: impact goes beyond technology to **people, policy and business processes.**
- ERP architecture must be **flexible** to support diverse hardware and software.
- Management must **filter out hyped technologies** that do not add value.

3.D Case study: a university ERP

- *Situation:* A university implements ERP for finance, HR and student fees.
- *Three-tier in action:* (1) Staff open the **web portal** in a browser (**web tier**). (2) The **application tier** runs fee-processing and reporting logic. (3) The **data tier** stores all student/finance data. (4) Because tiers are separate, they can add a second application server when users grow (**scalability**) but must pay for extra complexity (**limitation**).
- *SOA twist:* a “check student balance” **service** with a clear **contract** (inputs, outputs, errors) can be used by Library, Hostel and Finance applications.
- *Lesson:* tiers separate responsibilities; SOA separates **services**.

3.E Important comparisons (Chapter 3)

	Three-tier	SOA
Splits the system into	**Web / application / data tiers**	Small **services** (the business tier is broken into services)
Key idea	Layered separation	**Contract** between consumer and provider; **loose coupling**

	Server-centric	Client-centric
Strength	**Secure, no client software**	**Faster response**
Weakness	**Slower**	**Less secure**

3.F Exam-ready model answers (Chapter 3)

Q1. Describe the three-tier ERP architecture and its benefits. Web tier (browser portal), application tier (business processes and reporting, shields users from ERP internals), data tier (organizational data and relationships). Benefits: scalability, reliability, flexibility, maintainability, reusability, security. Limitation: expensive and complex.

Q2. What is SOA? Give benefits and drawbacks. SOA is a development model based on a **contract** between consumer and provider (functional description, inputs/outputs, pre- and postconditions, error handling). Benefits: loose coupling, location independence, works across platforms, composite applications, reliability, lower hardware cost. Drawbacks: costly, time-consuming, complex security/firewalls, inconsistent performance, needs enterprise-level focus.

Q3. What are Web services and which standards do they use? Interfaces that let different applications interact by exchanging messages, using open XML-based standards **WSDL, UDDI, SOAP**.

Q4. Benefits and drawbacks of cloud (as in this chapter). Use the table in 3.C-6.

3.G Chapter 3 recap

- ESA = **functional** (ERP modules) + **system** (hardware/software/network).

- Vendors: **SAP, Oracle/PeopleSoft, Microsoft Dynamics**.
- **3 tiers** (web, application, data) with 6 benefits and 1 limitation; **4th tier** for web services.
- **Server-centric** (secure, slow) vs **client-centric** (fast, less secure).
- **SOA**: contract-based services; **Web services**: WSDL, UDDI, SOAP, XML.
- Cloud: lease software; save licence costs; but depends on vendor and has security risks.

CHAPTER 4: Cloud Computing

(Chapter_4_-_Cloud_computing.pptx, 43 slides; slide title "What is Cloud Computing?")

4.A Chapter purpose

Teaches **what cloud computing is (NIST definition), its 5 essential characteristics, 4 deployment models, 3 service models, why businesses adopt it (cost and agility), how to decide and migrate, SLAs, multi-cloud, and an introduction to AWS.** This is the most list-heavy chapter, so it is high value for the exam.

4.B Key terminology

Term	Meaning (slides)
Data center	Facility housing and managing data: **hardware** (compute, storage, networks) plus **software** (apps, OS).
Cloud user	Person/organization requesting cloud resources (also **user, client, customer, consumer**).
Cloud Service Provider (CSP)	Organization providing a cloud platform, infrastructure, applications and services.
Multitenancy	**Multiple customers (tenants) of a CSP share resources without knowing about one another.**
Economies of scale	More production spreads costs over more units, so **per-unit cost falls**.

Term	Meaning (slides)
Organizational agility	Ability to **quickly adapt** to market/industry change.
Vendor lock-in	Reliance on **proprietary** technology that makes switching **hard and costly**.
Barrier to entry	Obstacles a new company faces entering a market.
CAPEX	**Capital expenses**: costs of **fixed assets** (purchase, maintenance, improvement). IT example: land, computing equipment, software.
OPEX	**Operational expenses**: **day-to-day running costs**. IT example: Internet costs, licensing fees.
TCO	**Total Cost of Ownership**: the complete cost of an object/service over its lifetime (purchase to disposal, direct and indirect).
Cloud-native	Designed specifically for the cloud environment.
Vertical scaling (scale up)	Adding resources (memory, processing power) **to the same node**.
Horizontal scaling (scale out)	Adding **more nodes** to a distributed system.
Diagonal scaling	Using **both** vertical and horizontal (slide footnote).
Monolithic application	Application designed as **a single unit**.
Microservices-based application	Broken into **many loosely coupled services** that interact.
Serverless	**Independent of the need to provision or manage servers.**
API	Application Programming Interface: lets **one piece of software talk to another**.
HTTP / JSON	HyperText Transfer Protocol / JavaScript Object Notation.
SLA	**Service Level Agreement**: contract stating the **uptime** and performance the CSP will support.
EC2	Amazon **Elastic Compute Cloud**; an EC2 **instance** = a **virtual machine** in AWS.

Term	Meaning (slides)
RDS	Amazon Relational Database Service (fully managed database).
NIST SP 800-145	The US standard that defines cloud computing.

4.C Detailed teaching notes

1. What is cloud computing?

Simple definition (slide 7): an **on-demand delivery of servers, storage, databases, applications and other IT resources as services over the Internet.** Resources are hosted in **large data centers** by CSPs (AWS, Microsoft Azure, Google) in locations around the globe; **implementation details are abstracted (hidden) from the user.** Structure: **front end** (user with browser/software) connects over the Internet to the **back end** (servers, storage, networks in the cloud).

Utility analogy (electricity/gas): plug in **on demand** (a vast grid managed by experts: low cost, reliable); **pay only for what you use** (metered); **use as much or as little** as you need.

2. NIST definition and model

NIST definition (learn this almost word-for-word): *"Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction."*

Common misconception (slide): cloud computing is **not a technology**; it is a **business or service model** that relies on **enabling technologies** such as **virtualization** and storage.

The NIST model has **5 essential characteristics, 4 deployment models, 3 service models** (the "5-4-3").

3. Five essential characteristics

#	Characteristic	Meaning	Mini-example
1	**On-demand self-service**	User **provisions resources automatically, with no human interaction** with the provider	Start a server yourself in minutes
2	**Broad network access**	Resources reachable from **many client platforms** (phones, tablets, laptops, workstations)	Open the same app on phone and laptop
3	**Resource pooling**	Provider **pools** servers, storage, etc. to serve **multiple consumers (multitenancy)**; location is abstracted (speaker note)	Many customers share one data center
4	**Rapid elasticity**	Resources **provisioned or released to match demand** (add VMs when demand grows, terminate when it drops)	Extra servers during exam-results day
5	**Measured service**	Usage is **monitored, controlled and reported**, giving transparency to both sides; billing by actual use: **"pay as you go" / pay-per-use**	Pay per hour of use

Memory aid: O-B-R-R-M → *On-demand, Broad access, Resource pooling, Rapid elasticity, Measured service* ("**O**ne **B**ig **R**ed **R**ocket **M**oves").

4. Four deployment models

A deployment model defines who the user of that cloud is and where the infrastructure resides.

Model	Who uses it	Where / who runs it	Strength	Watch-out	Example
Public	**General public**	On the **CSP's premises**	**Most cost-effective** (multitenancy, enormous scalability); pay metered charges (closest to utility model)	**No control of data/equipment location**: a concern under regulatory mandates	AWS, Google, Salesforce
Private	**One organization** (many departments/business units)	Owned and managed by the organization **on-premise** (cost-intensive); may be managed by a third party and hosted **off-premise**	**Better control** of data location and security	Needs **in-house IT skills**; **limited scalability**	Preferred by firms with an existing data center or legal/compliance needs (e.g., medical data, in-country data rules)
Hybrid	Public + private (or other) **bridged together**	Models stay **separate entities**, linked by standardized/proprietary technology for **data and application portability**	**Better alignment to business needs**	**Harder to manage**	Own data center plus public cloud for cheap backup
Community	**Several organizations with shared concerns** (mission, security, policy, compliance)	Run by the organizations **or a third party**; **on- or off-premise**	Shared concerns and costs		US government agencies; Salesforce CRM community cloud for Higher Education (speaker note)

Cloud bursting (hybrid): use **private cloud** to run applications; when **demand exceeds local capacity**, the **public cloud handles the burst.** *Slide chart:* on-premise capacity is flat at **50 units/month**; demand is about 40–55 most months but spikes to **80 (Feb), 115 (Mar), 120 (Apr)**; so the overflow goes to the public cloud. *Example:* a **tax preparation company** with **2–3× volume in tax season**: sizing on-premise for the peak leaves resources idle all year, so **size for typical load and burst to public cloud at peak.**

5. Three service models

A service model defines the type of service the provider offers. "As a service" means it is **delivered by way of software.**

	SaaS	PaaS	IaaS
Full name	Software as a Service	Platform as a Service	Infrastructure as a Service
What you get	**Ready-to-use applications** in the cloud	**Software platform** (languages, libraries, services, tools) to **build and deploy custom apps**	**Processing, storage, networks** and other basic computing resources
Main user	**End users and organizations**	**Developers**	**System administrators (IT Operations)**
Examples	**Salesforce CRM, Microsoft Office 365**	**Microsoft Azure, Google App Engine**	**Amazon EC2**
Key points	Cloud-native and **multitenant**; **prebuilt**, no significant customization; **CSP handles deployment, maintenance, patches, updates**; any device, anywhere; **most popular** service	Low-cost **rapid development** (web and mobile); mostly accessed via **APIs**; developers control apps and hosting settings but **not** networks, servers, storage, OS; **high risk of vendor lock-in**	Reduces/eliminates **capital cost** of owning hardware ("Why buy servers when you can rent them?"); **CSP manages infrastructure**, client manages OS and applications; **base layer** for PaaS and SaaS

Stack: SaaS on top of PaaS on top of IaaS (IaaS is the foundation).

SaaS provider duty (speaker note): ensure **one customer cannot access another's information. PaaS note:** leaders reduce lock-in by supporting **standard/open languages** (Google App Engine: **Java, Python, and Go**).

What is an API? (PaaS context): it lets **one piece of software talk to another**, defining how a programmer can **extend an existing application or build new ones. API using HTTP:** (1) client sends an **HTTP GET request** to an API endpoint asking for data in **JSON** → (2) if correctly formatted per the API documentation, (3) the server replies **HTTP 200 OK** with the JSON data.

6. Shared responsibility model

You always remain responsible for **your data**. Layers (top to bottom): **Data, Applications, Runtime, Middleware, OS, Virtualization, Servers, Storage, Networking.**

Layer	On-premise	IaaS	PaaS	SaaS
Data	You	You	You	**You**
Applications	You	You	You	CSP
Runtime / Middleware	You	You	CSP	CSP
OS	You	**You** (patching/upgrades)	CSP	CSP
Virtualization, Servers, Storage, Networking	You	CSP	CSP	CSP

(Table built from the slide statements: on-premise = organization manages everything; IaaS = CSP manages servers, storage, networking, virtualization while you manage OS, applications, data; "your duties shrink from IaaS to PaaS to SaaS, but you are always responsible for your data".) ⚠ **Check:** the exact split of runtime/middleware between you and the CSP in PaaS is **[Added]** from the standard model, because the slide's diagram colours were not readable in text.

7. Business drivers

Two major drivers: **Lower costs** and **Increased agility.** Hosting IT in-house is costly and distracts from **core competencies.**

- **Reducing costs: economies of scale** (public CSPs serve thousands of customers, so prices are reasonable; several pricing options by workload); **shift from CAPEX to OPEX** (no upfront hardware cost, except on-premise; **pay only for what you consume**).
- **Increasing agility: scalability and elasticity** (scale **up/down** and **out/in**, no guessing capacity); **faster time to market** (quickly deploy dev/test environments, experiment, innovate); **more focus on core business** (CSP does routine admin); **global access** (deploy in multiple regions for **lower latency**).
- **Capacity provisioning:** a **traditional data center** buys capacity in steps, leaving **waste** (idle capacity) and **customer dissatisfaction** (when demand exceeds capacity); **AWS** capacity **closely follows the demand curve: no waste, no dissatisfaction.**

8. Considerations for adoption

Choosing a deployment model by organization type:

Organization	Likely choice	Reason (slides)
Startups	**Public**	Greatest barrier is **capital**; lower cost outweighs other risks
Small-to-medium businesses	**Hybrid**	Mission-critical apps **private**, less critical apps **public**
Enterprises	**Private**	Large global base and financial strength; control **data location and security**

Migrate or retain?

- **Keep on-premise/private: mission-critical** apps; apps affected by **regulatory/compliance** limits on sensitive data; **legacy or proprietary-technology** apps; apps depending on a **private data center** (latency if moved).
- **Good fit for public cloud: non-critical, low-risk** apps: **collaboration** (email, web conferencing), **personal productivity** (word processing, spreadsheets), **development and testing.**

Choosing a cloud provider (5 questions): (1) How long in business? (2) Can it meet current **and future** needs? (3) How easy to **add/remove services**? (4) Can apps be **transferred to another provider** at contract end? (5) Does it **notify clients of maintenance windows** and is downtime scheduled?

Is there a cost advantage? Compare TCO.

On-premise TCO	Cloud costs
CAPEX: facilities, servers, storage devices, network equipment, operating systems	**Migration costs**
OPEX: power and cooling, personnel, Internet service	**Usage fees**
	Security and compliance costs

Free tools: **AWS TCO calculators and AWS Pricing Calculator.**

Migration strategies (4):

Strategy	Meaning	Speed / cost note	Example
Rehosting (lift-and-shift)	Redeploy on-premise apps **into cloud IaaS** unchanged	**Fastest**; **no optimization**; doesn't use cloud's full potential	AWS VM Import/Export (VM images → EC2)
Re-platforming	**Optimizations without changing core architecture**	Moderate	Move database to **Amazon RDS** (CSP handles admin)
Repurchasing	Switch to a **SaaS (cloud-native)** product	Frees IT staff from maintenance	HR system → **Workday**; Outlook → **Office 365**; in-house CRM → **Salesforce**
Rebuilding / re-architecting	**Change the core architecture** (monolith → **microservices** or **serverless**)	**Most expensive**; big gains in performance, scale, agility, features	

Memory aid for increasing effort/cost: *Rehost → Re-platform → Repurchase → Rebuild* ("**4 R's**").

Service Level Agreements (SLA): a **contract between customer and CSP** stating the **uptime** and **expected performance**; should say **how the CSP compensates** you if unmet (**money or service credits**); is **specific to a cloud service** (storage and VMs can have different SLAs); can be **uniform for all customers or customized**; helps ensure key resources stay available so organizations **avoid excessive losses.**

Example: AWS EC2 SLA. AWS makes **commercially reasonable efforts** to keep each EC2 instance available **at least 99.5%** of a monthly billing cycle. If not, you get a **service credit**:

Monthly uptime	Service credit
99.0% to < 99.5%	**10%**
95.0% to < 99.0%	**30%**
Below 95.0%	**100%**

Worked example: an instance is up 98.2% of the month → falls in 95.0% to <99.0% → **30% credit**. *(A month of 30 days has 720 hours; 99.5% uptime allows about 3.6 hours of downtime [Added calculation: 0.5% × 720 = 3.6 h].)*

Multi-cloud (current trend): using services from **more than one cloud vendor** (AWS, Azure, Google). **Benefits:** choose the **best-fit service** per vendor (performance, ease, cost, compliance) and **avoid vendor lock-in. Pitfall: management complexity.**

9. Amazon Web Services (AWS)

- Core services (compute, storage, database, networking, etc.) over the **AWS Global Infrastructure** of **Regions, Availability Zones and Edge Locations.**
- **Founded March 2006**; **world's largest public cloud provider** today (slide claim).
- **Three ways to access AWS: Management Console** (a GUI), **Command Line Interface (CLI)** (allows scripts), **SDKs** (preferred by developers; Python, Java, PHP, Ruby, C++ etc.).
- **Region:** a geographic area; each Region has **multiple physically separate Availability Zones, typically three.**
- **Availability Zone (AZ):** a **cluster of one or more discrete data centers**, **powered independently**, separated by about **100 km (60 miles)**, linked by **high-throughput metro fiber.** If one AZ suffers a power outage or disaster (earthquake, tornado), the others are **unaffected.**
- Diagram: *read bottom-up:* **Global Infrastructure → infrastructure building blocks (compute, storage, database, network) → cross-service features → tools to access AWS → your application on top.**

4.D Case study: a Kigali start-up and a bank

- **Start-up** (needs cheap fast launch): chooses **public cloud + SaaS/PaaS**; benefits from **CAPEX → OPEX**, no hardware purchase, **elasticity** at launch campaigns. Risk: vendor lock-in, data location.
- **Bank** (regulated customer data, existing data center): keeps core banking in a **private cloud**, uses the **public cloud for email and testing** (**hybrid**). Uses **cloud bursting** for seasonal spikes in mobile-money traffic **[Added example]**.
- *Lesson:* choose the deployment model by **capital, control, compliance and risk**, then pick a migration strategy by **speed vs benefit**.

4.E Important comparisons (Chapter 4)

	Public	Private	Hybrid	Community
Users	General public	One organization	Public + private bridged	Several organizations with shared concerns
Cost	Lowest (metered)	Highest (own infrastructure)	Mixed	Shared
Control	Least	Most	Mixed	Shared
Scalability	Huge	Limited	Good (bursting)	Depends

	CAPEX	OPEX
Nature	**Fixed assets** (buy and own)	**Day-to-day running**
IT example	Land, computing equipment, software	Internet, licensing fees
Cloud shift	**Reduced**	**Increased (variable, pay-per-use)**

	Vertical scaling	Horizontal scaling
Action	**Scale up**: add memory/CPU to the **same node**	**Scale out**: add **more nodes**

	Monolithic	Microservices	Serverless
Definition	**Single unit**	**Many loosely coupled services**	**No server provisioning or management** by you

4.F Exam-ready model answers (Chapter 4)

Q1. Define cloud computing (NIST) and list its five essential characteristics. Use the NIST definition above; list on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service, with one line each.

Q2. Compare SaaS, PaaS and IaaS with examples. *(Use the 3-column table in 4.C-5.)*

Q3. Explain the four cloud deployment models. Public (general public, CSP premises, cheapest, no location control), private (single organization, most control, needs skills, limited scalability), hybrid (bridged public+private, harder to manage; cloud bursting), community (organizations with shared concerns).

Q4. Why do businesses move to the cloud? Lower costs (economies of scale; CAPEX→OPEX; pay-per-use) and increased agility (elasticity, faster time to market, focus on core business, global access).

Q5. Describe the four migration strategies. Rehost (lift-and-shift, fastest, no optimization), re-platform (optimize without changing architecture, e.g., RDS), repurchase (move to SaaS, e.g., Salesforce), rebuild (re-architect, most expensive).

Q6. What is an SLA? A contract between customer and CSP specifying uptime and performance, with compensation (credits) if not met. Example: EC2 99.5% monthly uptime; 10% / 30% / 100% credits.

Q7. What is an Availability Zone and why does it matter? Cluster of one or more data centers in a Region, independently powered and about 100 km apart, so a disaster in one AZ does not affect the others.

4.G Chapter 4 recap

- Cloud = **business/service model**, not just technology.
- **5-4-3:** 5 characteristics, 4 deployment models (public, private, hybrid, community), 3 service models (SaaS, PaaS, IaaS).
- Drivers: **lower cost** (economies of scale, CAPEX→OPEX) and **agility**.

- Adopt carefully: deployment choice by org type, migrate vs retain, provider checklist, **TCO**, **4 migration strategies**, **SLA**, **multi-cloud**.
- AWS: Regions → Availability Zones (typically 3, ~100 km apart); access by Console, CLI, SDK.

CHAPTER 5: Data Warehouse, OLAP and OLTP

(Chapter_5_-_Data_Warehouse.pptx, 76 slides; no title slide: it starts with "Generic Warehouse Architecture". Slides 71–76 are references only.)

5.A Chapter purpose

Explains **how data from many operational systems is collected, cleaned and stored in a data warehouse, how it is analysed with OLAP (cubes and operations), how OLAP differs from OLTP, and how data marts and pitfalls fit in.**

5.B Key terminology

Term	Meaning (slides, in simple words)
Data warehouse (DW)	Central store of integrated, historical data for analysis and decisions (formal definition in Chapter 6: subject-oriented, integrated, time-variant, non-volatile).
Operational system	System running day-to-day business (source of "real-time data").
Informational system	System for analysis and decisions.
Derived data	Data **generated from existing data** using a mathematical operation or data transformation (summaries, calculated values).
Reconciled data	Cleaned/integrated data in the three-layer architecture.

Term	Meaning (slides, in simple words)
Data reconciliation	**Verification during data migration:** compare target data with source to make sure it was transferred correctly.
Metadata	Data about the data (a warehouse component).
Granularity	**Level of detail** in a table; **lower granularity = more detailed (low-level) information**.
ETL	**Extract, Transform, Load**: pull data from sources, transform it, load into the warehouse.
Refresh	Propagating updates from source data to the warehouse.
OLAP	**Online Analytical Processing**: approach to answering **multi-dimensional analytical (MDA) queries swiftly**.
OLAP cube / hypercube	Data structure optimized for very fast analysis; **numeric facts called measures categorized by dimensions**.
Measure / Dimension	Numeric fact (e.g., sales) / the way you categorize it (product, region, time).
Concept hierarchy	Levels within a dimension (e.g., day < month < quarter < year).
OLTP	**Online Transaction Processing**: operational systems handling day-to-day transactions.
Data mart	**Subset / departmental structure** of warehouse data used by OLAP; smaller, flexible, customized by department.
ROLAP / MOLAP / HOLAP / DOLAP / WOLAP / SOLAP	Relational / Multidimensional / Hybrid / Desktop / Web / Spatial OLAP (plus Mobile OLAP).

Term	Meaning (slides, in simple words)
FASMI	**Fast Analysis of Shared Multidimensional Information**: an alternative description of OLAP (Nigel Pendse and Richard Creath, *The OLAP Report*).
RDBMS	Relational Database Management System.

5.C Detailed teaching notes

1. Generic warehouse architecture

The diagram shows these parts: **Client(s)** doing **Query & Analysis** → the **Warehouse** (with **Metadata**) ← **Loading** ← **Integrator** ← several **Extractor/Monitor** units attached to the **data sources**; plus **Design phase**, **Maintenance** and **Optimization** functions.

```
Data sources --> [Extractor/Monitor] x n --> [Integrator] --> [Loading] -->
WAREHOUSE <--> Metadata
                                                                        |
                                              Clients <-- Query & Analysis <----+
   (Design phase, Maintenance, Optimization support the whole system)
```

Monitors detect changes at sources; **extractors/wrappers** pull the data; the **integrator** cleans and merges it.

2. Warehouse architectures: conceptual view

Architecture	Idea	Notes
Single-layer	**Every data element is stored once only**; operational data is used directly	A **virtual warehouse**
Two-layer	**Real-time data + derived data**	**Most commonly used in industry today**

Architecture	Idea	Notes
Three-layer	Adds **reconciled data** between real-time and derived data	Transformation of real-time data to derived data needs **two steps**: **data reconciliation**, then derivation; the top is the **view level** ("particular informational needs")

Three-layer picture, bottom to top: **Real-time data (operational systems) → Reconciled data → Derived data (physical implementation of the data warehouse) → View level (informational systems).**

3. Data quality: the reality

It is tempting to think building a warehouse is just **extracting operational data and entering it**: **"nothing could be farther from the truth."** Warehouse data comes from **disparate, questionable sources.**

Data integration across sources (example: a bank's Trust, Credit card, Savings and Loans systems):

- **Same name, different data**
- **Same data, different name**
- **Different keys, same data**
- **Data found in one place and nowhere else**

Data integrity problems (examples from slides): same person spelled differently (**Agarwal, Agrawal, Aggarwal**); multiple names for one company (**Persistent Systems, PSPL, Persistent Pvt. Ltd.**); different names for one place (**Mumbai, Bombay**); different **account numbers** for one customer; **required fields left blank**; **invalid product codes** at point of sale; **manual entry mistakes** ("in case of a problem use 9999999").

Data transformation example: applications A–D store **gender** as m/f, 1/0, x/y, male/female (**encoding**); pipeline length in cm, in, feet, yards (**unit**); balance called balance, bal, currbal, balcurr (**field name**). The warehouse must **convert all to one standard.**

4. Data transformation terms (learn all nine)

Term	Meaning
Extracting	**Capture data from the operational source "as is."**

Term	Meaning
Conditioning	**Convert data types** from source to target (the warehouse is **always a relational database**).
Scrubbing	Cleaning/correcting errors. ⚠ The slide defines scrubbing in the **hardware sense** (a background task that inspects memory/storage for errors and fixes them using checksums/copies). In warehousing it means **correcting dirty data**. Use the warehousing meaning unless asked for the slide's wording.
Merging	**Combine two or more similar records into one**: add variables, append cases, or **remove duplicates and incorrect information**.
Householding	**Identify all members of a household** (same address) so **only one mailing is sent per household**; can save a lot of money.
Enrichment	**Bring in data from external sources** to augment operational data.
Scoring	**Compute the probability of an event**, e.g., chance a customer will buy a new product.
Loading	Put the processed data into the warehouse.
Validating	**Check the accuracy and quality of source data before using/importing it**; a form of **data cleansing**.

Memory aid: *"Every Clever Student Must Have Excellent Study Lists, Very"* → Extracting, Conditioning, Scrubbing, Merging, Householding, Enrichment, Scoring, Loading, Validating.

5. Loading and refresh

Load issues: huge volumes; a **small time window** when the warehouse can be offline (usually **nights**); **when to build indexes and summary tables**; let administrators **monitor, cancel, resume and change load rates**; **recover gracefully** (restart from where you stopped, without losing **data integrity**).

Refresh = propagate source updates to the warehouse. Issues: **when** and **how** (techniques). **When to refresh: periodically** (every night/week) or after **significant events**; **on every update** only if users need **current data** (e.g., up-to-the-minute stock quotes); **policy set by the administrator** according to user needs and traffic; may differ per source.

Extraction and cleansing issues: sources of data; **data quality at sources**; merging sources; transformation; propagating updates; **terabytes** to load.

6. Granularity and derived data

- **Granularity** = level of detail in a table. **Summarized data cannot answer some questions.** *Example (slide):* "Did Anand call Seshadri last month?" **cannot be answered** if only **total call duration** per month is stored and not each call. But **detailed data is very voluminous.** Hence the idea of **dual levels of granularity**. ⚠ The slide says "the lower the granularity, the low-level information it contains": i.e., **low granularity = very detailed**.
- **Derived data** (calculated data) helps; **auxiliary views and indexes** speed queries. **OLAP services use SQL ROLLUP** to generate aggregate data in the warehouse.

7. Two distinct issues; issues in data warehousing

1. **How to get information into the warehouse** ("data warehousing").
2. **What to do with the data once it is there** ("warehouse DBMS").

Issues list: warehouse **design**; **extraction** (wrappers, **monitors/change detectors**); **integration** (cleansing and merging); **warehousing specification and maintenance**; **optimizations**; miscellaneous (e.g., **evolution**).

8. OLAP: Online Analytical Processing

Definition: approach to answering **multi-dimensional analytical (MDA) queries swiftly**; OLAP tools let users analyse multidimensional data **interactively from multiple perspectives.**

OLAP cube (hypercube): at the **core of OLAP**; a data structure **optimized for very quick analysis**; consists of **numeric facts (measures) categorized by dimensions.** OLAP databases are divided into **one or more cubes**, designed to make creating and viewing reports easy.

How it works (3 steps): (1) The data warehouse **extracts** information from many sources and formats (**text files, Excel sheets, multimedia files**); (2) the extracted data is **cleaned and transformed**; (3) it is loaded into an **OLAP server (or cube)** where information is **pre-calculated** for later analysis.

Operations: how many are there? ⚠ **The slides disagree:** slide 22 says **three** basic operations (**consolidation/roll-up, drill-down, slicing and dicing**); slide 27 says **four** (**roll-up, drill-down, slice and dice, pivot**); slide 44 says **five** (**roll-up, drill-down, slice, dice, pivot**). **Safest exam answer:** *list five: roll-up, drill-down, slice, dice, pivot* (and note that slice and dice are often grouped).

Operation	What it does	Slide example
Roll-up (consolidation)	**Aggregates** data: **climbing up a concept hierarchy** for a dimension, or **dimension reduction**	Hierarchy **street < city < province < country**; roll up from **city** level to **country** level. ⚠ The slide's last sentence ("grouped into cities rather than countries") is reversed; the result is grouped by **country**
Drill-down	**Reverse of roll-up**: **stepping down a concept hierarchy** or **introducing a new dimension**	Time hierarchy **day < month < quarter < year**; drill down from **quarter** to **month**
Slice	**Selects ONE dimension** to give a **new sub-cube**	**time = "Q1"**
Dice	**Selects TWO or more dimensions** to give a sub-cube	**location = Toronto or Vancouver; time = Q1 or Q2; item = Mobile or Modem**
Pivot (rotate)	**Rotates the axes** to give an **alternative presentation** of the data	Swap rows and columns (e.g., Region × Product ↔ Product × Region)

Memory aid: R-D-S-D-P → *"Real Dogs Slice Dice Pizza."* Roll-up goes **up/less detail**, drill-down goes **down/more detail**.

Example with a sales cube. Dimensions **Product, Region, Time**; measure **sales**. Roll-up from city to country shows national totals. Drill-down from quarter to month shows monthly sales. Slice time = Q1 shows only Q1's product × region table. Dice

Toronto/Vancouver, Q1/Q2, Mobile/Modem shows a small cube. Pivot swaps the product and region axes.

Multi-dimensional data (slide 62): a statement like "I sold $100M of goods" becomes useful when broken into **dimensions with hierarchical summarization paths:**

Product	Region	Time
Industry → Category → Product	Country → Region → City → Office	Year → Quarter → Month → Week → Day

Slide examples: products Juice, Cola, Milk, Cream, Toothpaste, Soap; regions W, S, N; months 1–7. **Telecomm slice example (slide 64):** a cube with **Product** (Household, Telecomm, Video, Audio), **Regions** (Europe, Far East, India) and **Sales channel** (Retail, Direct, Special): slicing on "Telecomm" gives one product's sub-cube. **Roll-up/drill-down (slide 65):** region hierarchy **Sales channel → Region → Country → State → Location address → Sales representative**; roll-up = higher aggregation, drill-down = low-level details.

FASMI: OLAP = **Fast Analysis of Shared Multidimensional Information.**

9. Types of OLAP systems

Type	Explanation (slides)
ROLAP (Relational OLAP)	**Extended RDBMS** with multidimensional mapping; **facts and dimension tables stored as relational tables**; works with data in a relational database; "fastest growing OLAP"
MOLAP (Multidimensional OLAP)	"Classical" OLAP using a **multidimensional data cube**; data is **pre-computed, re-summarized and stored** (major difference from ROLAP); **facts stored in multi-dimensional arrays**, dimensions used to query them
HOLAP (Hybrid OLAP)	**Aggregated totals in a multidimensional database; detailed data in a relational database**: gives ROLAP's data efficiency and MOLAP's performance

Type	Explanation (slides)
DOLAP (Desktop OLAP)	User **downloads part of the data to the desktop** and analyses it locally; **cheap to deploy**, **few functions**
WOLAP (Web OLAP)	OLAP **via a web browser**; **three-tier**: client, middleware, database server
Mobile OLAP	Access and analyse OLAP data on **mobile devices**
SOLAP (Spatial OLAP)	Manages **spatial and non-spatial data** in a **GIS**

ROLAP advantages: high data efficiency (query performance and access language optimized for multidimensional analysis); **scalability** for large and growing data. **ROLAP drawbacks: needs more resources** (people, software, hardware); **limits on aggregate calculations** (relies on SQL); **slow query performance compared with MOLAP.**

MOLAP architecture: database server, MOLAP server, front-end tool. *Flow:* the user requests a report through the interface → the **application logic layer of the MDDB** retrieves the stored data → it forwards the result to the user. *Example:* an accounting head runs the **profit and loss (P/L)** report for the corporation or one subsidiary; the MDDB retrieves **precompiled** figures. **MOLAP mainly reads precompiled data and is limited in creating new aggregations dynamically.**

HOLAP: mixture of ROLAP and MOLAP: **fast computation of MOLAP + scalability of ROLAP**; **two databases** (aggregated data in the OLAP cube, detail in the relational database). Benefits listed: **saves disk space**, stays **compact** (avoiding access-speed problems), **cube technology gives faster performance**, ROLAP's data is **instantly updated** so HOLAP users get **real-time data**, MOLAP **cleans and converts data**, giving **the best of both worlds**. ⚠ This slide's wording is partly muddled; learn "**best of both worlds**".

10. OLTP: Online Transaction Processing

Definition: an **operational system supporting transaction-oriented applications in a 3-tier architecture**; it **administers the day-to-day transactions of an organization**. Focus: **query processing, maintaining data integrity in multi-access environments**, and **effectiveness measured by transactions per second.**

Characteristics (slide list): small transactions with small amounts of data; **indexed data** is easy to access; **large number of users**; **fast, short response time**; databases directly accessible to end-users; **fully normalized schema** (consistency); **only predefined operations on a few records**; stores records of **the last few days or a week**; supports complex data models and tables. ⚠ "Stores last few days or a week" and "complex data models" sound inconsistent with other slides (the comparison table says OLTP data is **current, isolated**); give both only if asked for slide wording.

OLTP in the warehouse picture (slides 50–51): business/enterprise strategy (set by top management) → **business processes** (activities achieving a goal) → **customers, orders and products** stored in the OLTP database (customers, suppliers, employees, orders) → **ETL** (extracts from RDBMS sources, **transforms** with concatenations and calculations, **loads** into the warehouse) → **data mart and data warehouse** (the data mart is a structure/access pattern used by OLAP to store processed data) → **data mining, analytics and decision making** (discover patterns, analyse raw data, make decisions for growth).

OLTP examples: ATM (main example), online banking, online airline ticket booking, sending a text message, order entry, adding a book to a shopping cart.

OLTP vs OLAP summary diagram (slide 53): *Business processes* (**OLTP operations**) produce **master data and transactions** that flow into the *Business data warehouse* (**OLAP information**); **analytics** from the warehouse feed back into **business strategy**, which guides the business processes.

OLTP vs OLAP table (image on Chapter 6, slide 55):

	OLTP	OLAP
User	**Clerk / IT professional**	**Knowledge worker**
Function	**Day-to-day operations**	**Decision support**
Database design	**Application-oriented**	**Subject-oriented**
Data	**Current, isolated**	**Historical, consolidated**
View	**Detailed, flat relational**	**Summarized, multidimensional**
Usage	**Structured, repetitive**	**Ad hoc**
Access	**Read/Write**	**Read** (mostly)

Memory aid: OLTP = "Operations, **L**ots of small writes, **T**ransactions, **P**eople-clerks"; **OLAP = analysis**.

11. Data warehouse vs data marts

- **Data warehouse: organizationally structured, atomic, detailed** data; **normalized**; more history.
- **Data mart: departmentally structured, lightly summarized**; serves OLAP. Characteristics: **small, flexible, customized by department; source is the (departmentally structured) data warehouse.**
- **Techniques for creating a departmental mart (slide 58): subset, summarized, superset, indexed, arrayed** (for departments such as **Sales, Finance, Marketing**).
- **Data-mart-centric approach:** marts are built straight from data sources. **Problem: if you end up creating multiple warehouses, integrating them is a problem.** *(Slide asks: "What comes first?" the warehouse or the marts.)*

12. Data warehouse pitfalls (from slides)

1. You will spend **much time extracting, cleaning and loading** data.
2. You will find **problems with the systems feeding** the warehouse.
3. You will need to **store data not captured by any existing system**.
4. You will need to **validate data not validated by transaction systems**.
5. Some feeding systems **won't contain detail**.
6. Many users are trained but **seldom use** their training.
7. Users will develop **conflicting business rules**.
8. **"Overhead" eats disk space**.
9. **Load time expands** to fill the available window.
10. **Security** cannot be assigned with a transaction-processing mindset.
11. It is a **HIGH-maintenance system**.
12. You fail if you focus on **resource optimization** and neglect **project, data and customer management** and what **adds value to the customer**.

Research issues (DW and OLAP): data cleaning (inconsistencies, data mining techniques); **physical design** (summary tables, partitions, indexes); **query processing** (choose summary tables, dynamic optimization, cost estimation, splitting work between

OLAP server and back end); **warehouse management** (detect **runaway queries**, resource management, **incremental refresh**, failure recovery during load/refresh, scheduling, workflow technology).

5.D Case study: a university data warehouse

- *Situation:* The registrar (student records), finance (fees) and library (loans) each run **OLTP** systems. Management wants to know whether late fee payment affects exam results.
- *Steps:* (1) **Extract** from each system "as is"; (2) **condition/scrub** (gender coded m/f vs 1/0; "Huye/Butare" naming), **merge** duplicate student records; (3) **load** at night into the **warehouse**; (4) build a **cube**: measure = average mark; dimensions = **fee status, department, semester**; (5) analysts **roll up** to faculty, **drill down** to department, **slice** on semester = 1, **pivot** to compare.
- *Lesson:* OLTP runs the university; OLAP/warehouse helps decide.

5.E Important comparisons (Chapter 5)

	ROLAP	MOLAP	HOLAP
Storage	**Relational tables**	**Multidimensional arrays (cube)**	**Aggregates in cube + detail in relational DB**
Data	Detailed, scalable	**Pre-computed**	Both
Speed	**Slower** queries	**Fast**	Fast + scalable
Weakness	Resource-hungry, SQL aggregate limits	**Cannot easily compute un-stored aggregates**	Complexity ⚠

	Slice	Dice
Dimensions selected	**One**	**Two or more**

	Roll-up	Drill-down
Direction	**Up** hierarchy / reduce dimensions	**Down** hierarchy / add dimension

	Roll-up	Drill-down
Detail	Less	More

	Data warehouse	Data mart
Scope	Whole organization	One department
Detail	**Atomic, detailed**	**Lightly summarized**

5.F Exam-ready model answers (Chapter 5)

Q1. Define OLAP and describe the cube. OLAP is an approach to answering multi-dimensional analytical queries swiftly. The OLAP cube (hypercube) is a data structure optimized for fast analysis, holding numeric facts (measures) categorized by dimensions.

Q2. Explain the OLAP operations with examples. Use the table in 5.C-8.

Q3. Compare ROLAP, MOLAP and HOLAP. Use 5.E; add one advantage and drawback for each.

Q4. Differentiate OLTP and OLAP. Use the 7-row table.

Q5. List and explain data transformation terms. Give the nine terms and one-line meanings.

Q6. Discuss problems in building a data warehouse. Data quality and integration problems (same name/different data, spelling variants), load windows, refresh policy, granularity, and the pitfalls list.

Q7. Explain the three-layer warehouse architecture. Real-time data → reconciled data → derived data → view level; two-layer is most common; single-layer is a virtual warehouse.

5.G Chapter 5 recap

- Warehouse components: extractors/monitors, integrator, loading, warehouse, metadata, query & analysis.
- Architectures: **single (virtual) / two (most common) / three (reconciled)**.
- **9 transformation terms**; load + refresh issues; granularity trade-off.

- **OLAP:** cube = measures × dimensions; ops **roll-up, drill-down, slice, dice, pivot**; types **ROLAP, MOLAP, HOLAP**, plus DOLAP, WOLAP, Mobile, SOLAP.
- **OLTP** = day-to-day transactions; **OLAP** = analysis.
- **Data mart** = departmental subset; data-mart-centric approach causes integration problems.
- 12 pitfalls.

CHAPTER 6: Decision Making and Business Intelligence (BI)

(Chapter_6_-_BI_ADS.pptx, 65 slides; title "Business Intelligence Overview", Dr. James Rwigema)

6.A Chapter purpose

Starts with **how decisions are made** (the 8-step process, levels of decisions, the data → information → knowledge → decision → action cycle), then defines **Business Intelligence**, its **components (data warehouse, data mining, business analyst), layers, architecture, framework, benefits and business value.** BI is where Chapters 1–5 come together: data stored in a warehouse is analysed (OLAP, data mining) to support decisions.

6.B Key terminology

Term	Meaning (slides)
Decision	**Making a choice from two or more alternatives.**
Decision making	**The process of examining your possibilities and options, comparing them, and choosing a course of action.** (General: making choices by identifying a decision, gathering information and assessing alternatives.)
Problem	**A discrepancy between an existing and a desired state of affairs.**

Term	Meaning (slides)
Decision criteria	Factors **important (relevant) to resolving the problem**.
Structured / semi-structured / unstructured decision	Routine and well-defined / partly defined / non-routine, long-term.
Business Intelligence (BI)	Skills, processes, technologies, applications and practices **used to support decision making**; turns raw data into meaningful information.
Business analytics	**Tools to explore past data to gain insight into future business decisions.**
Big data	**Data sets so large or complex that traditional data processing applications are inadequate.**
Data mining	**Tools for discovering patterns in large data sets** (computational process at the intersection of **AI, machine learning, statistics and database systems**).
Data warehouse	**A subject-oriented, integrated, time-variant and non-volatile collection of data in support of the management decision-making process.**
DSS	Decision Support System (a BI system is also called a DSS).
Data mashup	**Integration of two or more data sets in a single graphical interface.**
Dashboard	A prime example of a data mashup; pulls data from multiple, disparate sources.
ETL	Extract, Transform, Load.
ODS	Operational Data Store (in the BI architecture diagram).
TDWI	The Data Warehousing Institute (source of the BI business-areas chart).

Term	Meaning (slides)
Single version of truth	One consistent set of figures used across the organization (a BI benefit).

6.C Detailed teaching notes

Part A: The decision-making process

Why it matters: decision making is a daily activity; in business it is a **habit and a process**. **Effective decisions make profit; unsuccessful ones cause losses**, so corporate decision making is **the most critical process** in an organization. In decision making we **choose one course of action from several alternatives**, may use many tools and techniques, and may decide **privately or collectively**. It is usually **hard**: most corporate decisions involve **some dissatisfaction or conflict with another party. Every decision-making process produces a final choice; the output can be an action or an opinion of choice.**

The 8 steps of decision making (learn in order):

#	Step	Key points (slides)
1	**Identification of the problem**	A problem is a **discrepancy between existing and desired state.** Cautions: (a) make sure it's a **problem and not just a symptom**; (b) identification is **subjective**; (c) discrepancies are found by **comparing current results with a standard**; (d) managers won't call a discrepancy a problem if they think they lack the **authority, information or resources** to act
2	**Identification of decision criteria**	Factors important to resolving the problem, e.g., **costs** (investment required), **risk** (chance of failure), **outcomes desired** (growth of the firm)
3	**Allocating weights to the criteria**	Criteria are **not equally important**; a **weight** puts them in **priority order**; each solution has pros and cons; the manager **scores** criteria according to the situation

#	Step	Key points (slides)
4	**Developing alternatives**	Select **viable alternatives** that could resolve the problem; **just list, do not evaluate yet**
5	**Analysis of alternatives**	**Appraise each alternative's strengths and weaknesses** against the criteria and weights from steps 2 and 3
6	**Selection of an alternative**	Choose the best, i.e., the one with the **highest total score** from step 5
7	**Implementation**	Put it into action by **conveying it to those affected and getting their commitment**
8	**Evaluation of decision effectiveness**	Check whether the problem was resolved; if it **persists**, **re-assess the problem and start over**

Memory aid: "P-C-W-A-A-S-I-E" → *Problem, Criteria, Weights, Alternatives, Analyse, Select, Implement, Evaluate* ("**P**eople **C**an **W**in **A**ll **A**rguments **S**ensibly **I**f **E**ducated").

Simplified view (Fig. 1, three gears): Problem identification → Finding alternatives → Making a choice. Quote on slide: **"Information and knowledge form the backbone of the decision-making process."**

Worked example (weighted scoring). [Added] The slides say to weight criteria, score each alternative and choose the highest total; they give no numbers, so here is an illustration. Choose between **Cloud (A)** and **On-premise (B)**. Weights: cost 0.5, risk 0.3, growth 0.2 (sum 1.0). Scores out of 10:

	Weight	A: Cloud score	A weighted	B: On-premise score	B weighted
Cost	0.5	9	0.5 × 9 = 4.5	4	0.5 × 4 = 2.0
Risk	0.3	6	0.3 × 6 = 1.8	8	0.3 × 8 = 2.4
Growth	0.2	8	0.2 × 8 = 1.6	5	0.2 × 5 = 1.0
Total	1.0		**7.9**		**5.4**

Formula: *Total score = Σ (weight × score)*. A scores highest, so **choose A** (step 6).

Who makes decisions? (Fig. 2)

Level	Decision type	Maker	Time effect
Operational	**Structured**	**Lower management**	**Daily activities, short-term**
Tactical	**Semi-structured**	**Middle management**	Medium
Strategic	**Unstructured**	**Top management**	**Long-term effect**

Action cycle of decision making (Fig. 3: Data–Information–Knowledge–Decision Making–Action cycle). The user **applies intelligence (analysis) to data** to produce **information**; information is **the basis of knowledge**; knowledge is **used in decision making**; decisions **trigger actions**; actions **generate more data**, and the cycle repeats.

```
DATA --(analysis)--> INFORMATION --> KNOWLEDGE --> DECISION --> ACTION
  ^                                                              |
  +---------------------- (actions generate more data) ------------+
```

Characteristics of data for good decision making (6): Accuracy (accurate enough for the purpose), **Validity** (complies with requirements and definitions), **Reliability** (collection is consistent over time and across sources), **Timeliness** (captured quickly after the event and available when needed), **Relevance** (relevant to the intended purpose), **Completeness** (watch for missing, invalid and incomplete data).

Memory aid: "A VRTRC" → *Accuracy, Validity, Reliability, Timeliness, Relevance, Completeness.*

Conclusion on slides: weigh positive and negative consequences and favour positive outcomes; this avoids losses and sustains growth.

Part B: Business Intelligence

Business analytics vs BI vs big data vs data mining:

Term	What it is
Business analytics	Tools to explore **past data** to gain insight for **future** decisions
BI	**Tools and techniques to turn data into meaningful information**

Term	What it is
Big data	Data sets **so large/complex that traditional applications are inadequate**
Data mining	Tools for **discovering patterns in large data sets**

BI overview diagram (Fig. 4): Data sources → Data warehouse → Analytical tools (OLAP, data mining, data visualisation) → Analysis results (visualisation).

Definitions of BI (learn two):

1. **BI = collecting and refining information from many sources (internal and external), analysing and presenting it in useful ways (dashboards, visualizations), so people can make better decisions that help build and retain competitive advantage.**
2. **BI refers to skills, processes, technologies, applications and practices used to support decision making.** Others: *the set of theories, methodologies, architectures, techniques and tools for transforming raw data into meaningful information for business analysis*; *applications and technologies transforming business data into action*; *technology for understanding the past and predicting the future*.

Summary on BI (slides 31–34): computer-based techniques for **spotting, digging out and analysing business data** (e.g., sales revenue by product/department, costs and incomes); gives **historical, current and predictive views** of operations; aims at **better decision making**; BI is a **process and technology** used to control huge amounts of data and mine it to make decisions more efficient; it gives an overview **from the past, present to future predictions**; because of this, **a BI system is also known as a Decision Support System (DSS)**; BI enables **intelligent, fact-based decisions.**

A simple BI system has three components:

1. **Data warehouse:** comprehensive data of the firm.
2. **Data mining:** techniques to exploit data and discover information: **classification, clustering, association rules, prediction.**
3. **Business analyst:** leaders of enterprises who make **strategic decisions.**

BI process: the key aspects are the **data warehouse and data mining**, because data in BI is **synthesized data**. The warehouse is characterized by **various sources and formats** and is **clustering and historical** ⚠ (slide wording; read as: **integrated and historical**). Analysis in BI is **not just query and filtering**; it uses **data mining** for **classification, clustering or prediction.** So **BI is tightly linked to the data warehouse and data mining.**

BI and analytics (Gartner, 2014): capability list. The slide groups capabilities under three headings **Information delivery, Analysis, Integration** (plus BI infrastructure and administration). Items listed: **reporting, interactive visualization, dashboards, search-based data discovery, Microsoft Office integration, ad hoc report/query, geospatial and location intelligence, mobile BI, OLAP, embedded advanced analytics, collaboration, metadata management, business-user data mashup and modeling, embeddable analytics, development tools, support for big data sources.** ⚠ The text boxes are jumbled, so **I cannot confirm which item belongs under which heading**; learn the headings and a handful of items.

Evolution of BI (diagram, x-axis = analytics maturity, y-axis = competitive advantage). Stages from low to high: **Raw data → Cleaned data → Standard reports → Ad hoc reports and OLAP → Generic predictive analytics → Predictive modeling → Optimization.** Questions answered: **"What happened?"** (reports and OLAP), **"Why did it happen?"**, **"What will happen?"** (predictive modeling), **"What is the best that could happen?"** (optimization). Left half: **Sense & Respond**; right half: **Predict & Act.**

Why is BI important / business areas. TDWI chart (482 respondents): business areas addressed by BI application packages: **Finance/accounting 70%, Sales 56%, Marketing 45%, Customer service 37%, Order management 34%, Human resources 31%, Procurement 26%, Supply chain management 23%, Manufacturing 17%, Other 11%.**

BI strategy: improve the organization by giving **business insights to all employees** for **better, faster, more relevant decisions**: let business users **query and write reports**; **simplify reporting across several transaction systems**; **store historical data longer** than a transaction system would.

Why companies need BI (4 levels of questions):

Level	Question answered
Standard reports	**What happened?**
Ad hoc reports	**How many, how often, where?**
Query / drill-down	**Where exactly is the problem?**
Alerts	**What actions are needed?**

Part C: BI components

- **Data warehouse (DW): subject-oriented, integrated, time-variant, non-volatile** collection of data supporting management decisions. *In simple words:* **subject-oriented** = organized by business subject (customers, sales); **integrated** = from many sources, made consistent; **time-variant** = keeps history over time; **non-volatile** = data is stored, not changed or deleted by daily transactions. *(Meaning of each word is **[Added]** explanation; the one-line definition is on the slide.)*
- **OLTP:** a class of information systems that **facilitate and manage transaction-oriented applications, typically data entry and retrieval**; increasingly supports transactions that **span a network and may involve more than one company**; new OLTP software uses **client/server processing and brokering software** so transactions can run on **different computer platforms** in a network.
- **OLAP:** approach for answering **multi-dimensional analytical queries swiftly**; users analyse data from multiple perspectives; operations **consolidation (roll-up), drill-down, slicing and dicing**; **consolidation = aggregation of data accumulated and computed in one or more dimensions.**
- **OLAP vs OLTP:** see the table in Chapter 5, section 10 (the same image is used here).
- **Data mining and its process:** computational process of **discovering patterns in large data sets** using **AI, machine learning, statistics and databases. Process (diagram): Pre-processing → Data mining → Result validation.** *Pre-processing:* a **target data set must be assembled** before algorithms can run. *Data mining:* **anomaly detection, clustering, classification, regression, summarization.** *Result validation:* final step is to **verify that the patterns found also occur in the wider data set.**

Part D: BI layers, architecture, framework

BI layers (diagram), top to bottom:

1. **Presentation layer: reporting** and **analysis**.
2. **Data warehouse layer:** the **data warehouse**.
3. **Source layer: Finance dept., HR dept., CRM, OS (operational systems), Excel** and other sources.

BI architecture (diagram), top to bottom: Operational systems → ETL (with metadata) → Operational Data Store (ODS) → Enterprise Data Warehouse (with metadata) → Line-of-business data marts. Data marts are of two kinds: **dependent data marts** (fed from the enterprise data warehouse) and **independent data marts** (loaded directly via ETL, bypassing the warehouse).

BI framework (diagram): Getting data in (data warehousing): data sources → **ETL** → **data warehouse + metadata** → **Getting data out (business intelligence): data access** through **data marts** to users (analysts, query/reporting, OLAP cubes, charts).

BI types: the slide titled "BI Types" contains **only a picture**, which I could not read: **not covered, please check your original slide.**

BI as a framework (slides 61–62): BI is a **conceptual (umbrella) framework for decision support** that **combines architecture, databases (or data warehouse), analytical tools and applications. Objective: interactive access to data**, **manipulation of data**, and enabling **business managers to make good decisions. Process: data → information → decisions → actions.**

Benefits of BI (10): time savings; single version of truth; improved strategies and plans; improved tactical decisions; more efficient processes; cost savings; faster, more accurate reporting; improved decision making; improved customer service; increased revenue. "Many benefits are intangible."

The business value of BI: *How BI can help:* **assess readiness** for new business realities; take a **holistic approach** to BI functionality; **leverage best practices and anticipate hidden costs.** *Key questions for analysis:* How can enterprises **maximize BI investments**? **What BI functionality** do they need and use today? What are the **hidden costs** of BI initiatives?

6.D Case study: BI for a retail chain

- *Situation:* A supermarket chain in Rwanda has daily sales in OLTP systems at each branch.
- *Steps:* (1) **Source layer:** branch POS, finance, CRM. (2) **ETL** cleans and loads into the **data warehouse**. (3) **Data marts** for Sales and Marketing. (4) **OLAP:** roll up sales by province, drill down to branch, slice by month. (5) **Data mining: clustering** customers into groups; **classification** of likely loyalty-card churners; **prediction** of next month's demand. (6) **Presentation layer:** dashboard for managers. (7) Managers decide (e.g., restock) → action → new sales data.
- *Concept applied:* **data → information → knowledge → decision → action** cycle; **"What happened / where is the problem / what action?"** levels.
- *Lesson:* BI = warehouse + OLAP + data mining + presentation, serving decisions.

6.E Important comparisons (Chapter 6)

	OLTP	OLAP
Purpose	Day-to-day operations	Decision support
Data	Current, isolated	Historical, consolidated
Users	Clerks / IT staff	Knowledge workers

	Business analytics	BI	Big data	Data mining
Focus	Explore **past** data for **future** decisions	Turn **data into information**	**Huge/complex** data sets	**Discover patterns**

	Operational	Tactical	Strategic
Decision type	Structured	Semi-structured	Unstructured
Made by	Lower mgmt	Middle mgmt	Top mgmt

	Dependent data mart	Independent data mart
Source	**Enterprise data warehouse**	**Directly from ETL / sources**

6.F Exam-ready model answers (Chapter 6)

Q1. Describe the 8 steps of decision making. *(List with one line each, as in the table.)*

Q2. Define BI and explain its components. BI is skills, processes, technologies, applications and practices to support decision making by turning raw data into meaningful information. Components: data warehouse, data mining (classification, clustering, association rules, prediction), business analyst.

Q3. Explain the BI layers/architecture. Source layer (finance, HR, CRM, Excel) → ETL → (ODS) → data warehouse → data marts → presentation layer (reporting and analysis).

Q4. Distinguish OLTP and OLAP. Use the table.

Q5. List benefits of BI. Time savings, single version of truth, better strategies and tactical decisions, efficiency, cost savings, faster and more accurate reporting, better customer service, increased revenue; many are intangible.

Q6. What are the characteristics of good data for decision making? Accuracy, validity, reliability, timeliness, relevance, completeness.

6.G Chapter 6 recap

- **Decision = choice among alternatives; 8 steps;** levels: operational / tactical / strategic.
- **Data → Information → Knowledge → Decision → Action** (cycle).
- **BI** = process + technology that turns data into information for decisions; **BI system = DSS.**
- Components: **DW + data mining + business analyst.** Data mining process: **pre-processing → mining → validation.**
- Layers: **source → data warehouse → presentation.**
- 4 BI question levels: **standard reports, ad hoc, drill-down, alerts.** 10 benefits.

PART III: HIGH-VALUE REVISION SHEETS

A. Definitions sheet

Term	One-line definition
Distributed system	Independent computers that appear to users as a **single coherent system**, coordinating **only by passing messages**.
Transparency	**Hiding** that processes and resources are physically distributed.
Openness	Services offered via **standard interfaces** (IDL) so systems interoperate and are portable.
Scalability	Ability to grow in **size, geography, and administrative domains**.
Middleware	Layer **between OS and applications** that connects components across a network.

Term	One-line definition
NOS / DOS	Own OS per computer, share files / OS of each node knows the others.
Client–server / P2P	Clients request from servers / equal peers share resources with **no central coordination**.
Mobile agent	Running program (code + data) that **travels between computers** doing a task and returns results.
Silo	**Compartmentalized unit isolated from its environment.**
BPR	Redesign of work as **cross-functional processes**.
Logical / physical integration	Share data and change organization / connect heterogeneous systems.
ERP	**Integrated, multi-module** software package supporting **several business functions**.
Three-tier	**Web tier, application tier, data tier.**
SOA	Software model based on a **contract** between consumer and provider (services, loose coupling).
Web service	Interface letting applications interact **by sending/receiving messages** (WSDL, UDDI, SOAP, XML).
Cloud computing (NIST)	**Model for ubiquitous, convenient, on-demand network access to a shared pool of configurable resources**, rapidly provisioned with minimal management effort.
Multitenancy	Many customers **share resources** without knowing each other.
CAPEX / OPEX / TCO	Fixed-asset costs / day-to-day running costs / **complete lifetime cost**.
SLA	Contract setting **uptime/performance** and **compensation**.

Term	One-line definition
Availability Zone	Cluster of **one or more data centers**, independently powered, in a Region.
Granularity	Level of **detail** in data (low granularity = more detail).
ETL	**Extract, Transform, Load.**
OLAP	Approach for **swift multidimensional analytical queries**.
OLAP cube	Optimized structure of **measures** categorized by **dimensions** (hypercube).
OLTP	**Day-to-day transaction** processing system.
Data mart	**Departmental, lightly summarized** subset used by OLAP.
Data warehouse	**Subject-oriented, integrated, time-variant, non-volatile** data for decision making.
Decision	**Choice** among two or more alternatives.
Problem	**Discrepancy** between existing and desired state.
BI	Skills, processes, technologies and practices that **turn data into information to support decisions.**
Data mining	Discovering **patterns in large data sets** (AI, ML, statistics, databases).
Data mashup	Combining **two or more data sets** in one interface (dashboard).

B. Comparison sheet

Pair	Difference in one line
Multiprocessor vs multicomputer	Shared memory + memory read/write vs separate memory + message passing

Pair	Difference in one line
Message passing vs shared memory	Separate address spaces vs shared address space (lower latency)
NOS vs DOS vs middleware	NOS: independent OSs, files; DOS: OSs know each other, rarely used; Middleware: layer above OSs, OSs can differ
Client–server vs P2P	Distinct roles vs equal roles, no central server
RPC vs RMI	Remote procedures as if local vs the same idea for distributed objects
Direct vs indirect communication	Sender knows receiver vs space and time uncoupled
Migration vs relocation transparency	Hidden from the object vs hidden from a client in use
Horizontal vs vertical silo	By function vs by management level
Logical vs physical integration	Process/data sharing vs technical connectivity
Server-centric vs client-centric	Secure but slower vs faster but less secure
Three-tier vs SOA	Layers (web/app/data) vs contract-based services
Public vs private cloud	General public, CSP premises, cheapest vs one organization, more control, costly
Hybrid vs community cloud	Public+private bridged vs several organizations with shared concerns
SaaS vs PaaS vs IaaS	Apps (end users) vs platform (developers) vs infrastructure (sysadmins)
CAPEX vs OPEX	Buy assets vs pay running costs (cloud shifts CAPEX → OPEX)
Vertical vs horizontal scaling	Bigger node vs more nodes
Rehost vs re-platform vs repurchase vs rebuild	Lift-and-shift vs optimize without redesign vs buy SaaS vs re-architect

Pair	Difference in one line
Roll-up vs drill-down	Less detail (up the hierarchy) vs more detail (down)
Slice vs dice	One dimension vs two or more
ROLAP vs MOLAP vs HOLAP	Relational tables vs multidimensional cube vs both (aggregates in cube, detail relational)
OLTP vs OLAP	Operations, current, read/write vs decisions, historical, mostly read
Data warehouse vs data mart	Organization-wide, atomic vs department-level, lightly summarized
Dependent vs independent data mart	From the warehouse vs loaded directly from ETL
Business analytics vs BI vs data mining vs big data	Past data for future decisions / data→information / pattern discovery / huge complex data
Operational vs tactical vs strategic decisions	Structured, lower mgmt / semi-structured, middle / unstructured, top

C. Processes and classifications sheet

Chapter 1: 8 transparency types · 3 goals · 3 scalability dimensions · 4 scaling techniques · 8 challenges · 8 false assumptions · 3 hardware concepts · 3 software concepts · 4 middleware service groups · 4 architectural questions · 3 communication paradigms · 2 DS types.

Chapter 2: 8 steps of integration (resource categorization → compliance/standards → legacy support → middleware → authentication/authorization → centralized IT services → back-up/recovery/security → hardware/software standardization) · 4 benefits · 4 limitations.

Chapter 3: 3 tiers (web, application, data) + 4th web-services tier · 6 benefits of 3-tier · SOA contract (5 items) · Web-service standards (WSDL, UDDI, SOAP).

Chapter 4: 5 characteristics · 4 deployment models · 3 service models · 2 business drivers · 4 migration strategies · 5 provider questions · EC2 SLA credits (10/30/100%) · access to AWS (Console, CLI, SDK).

Chapter 5: 3 warehouse architectures · 9 transformation terms · OLAP 5 operations · OLAP types (ROLAP, MOLAP, HOLAP, DOLAP, WOLAP, Mobile, SOLAP) · mart techniques (subset, summarized, superset, indexed, arrayed).

Chapter 6: 8 decision steps · 3 decision levels · action cycle (data → information → knowledge → decision → action) · 6 data characteristics · 3 BI components · 4 BI question levels · 3 BI layers · data mining process (pre-processing → mining → validation) · 10 BI benefits.

D. Formula and calculation sheet

The notes contain **no mathematical formulas**; calculation-type questions would most likely be these:

Calculation	Formula	Variables	Worked example
Weighted decision score (Ch6) **[Added structure]**	Total = Σ (weight × score)	weight = importance (weights sum to 1); score = rating of alternative	Weights 0.5/0.3/0.2, scores 9/6/8 → 4.5 + 1.8 + 1.6 = **7.9**
EC2 SLA credit (Ch4, slide table)	Look up uptime in table	uptime % in the month	98.2% → 95.0%–<99.0% → **30%** credit; 99.7% → no credit; 93% → **100%**
Allowed downtime (Ch4) **[Added]**	Downtime = (1 − uptime) × hours in month	30 days = 720 h	(1 − 0.995) × 720 = **3.6 h**
Speed of a DS (Ch1 slide example)	Total power = CPUs × MIPS each	10,000 CPUs, 50 MIPS	10,000 × 50 = **500,000 MIPS**

Calculation	Formula	Variables	Worked example
TCO comparison (Ch4, concept only)	On-premise TCO = CAPEX + OPEX; cloud cost = migration + usage fees + security/compliance		Compare totals; no numbers on slides

E. Common mistakes and confusions

1. **Cloud is a business/service model, not a technology** (NIST misconception slide).
2. **Public cloud is *not* automatically "better"**: private gives more control; hybrid is harder to manage.
3. **SaaS is for end users, PaaS for developers, IaaS for system administrators**: do not swap them.
4. **You are always responsible for your data**, even in SaaS.
5. **Rehosting is the fastest but not the most optimized; rebuilding is the most expensive.**
6. **Slice = one dimension; dice = two or more.** Do not mix them up.
7. **Roll-up goes up the hierarchy** (city → country); the slide's wording that it groups "into cities" is wrong.
8. **OLAP operations count:** answer with five (roll-up, drill-down, slice, dice, pivot).
9. **OLTP ≠ OLAP:** OLTP is write-heavy and current; OLAP is read-heavy and historical.
10. **Low granularity = high detail.**
11. **Transparency cannot be complete**: failures cannot be fully hidden.
12. **Openness vs transparency** are different goals.
13. **Migration vs relocation** transparency.
14. **Middleware sits above the OS**, not inside it.
15. **Chapter 2 and 3 duplicate slides:** do not study ESA twice.
16. **Time uncoupling** (not "time coupling") in indirect communication.
17. **Data scrubbing in the warehouse** = cleaning data, even though the slide gives a memory-hardware definition.
18. **"Information" vs "knowledge"** in the action cycle: information is the basis of knowledge.

F. Memory aids (collected)

Topic	Aid
DS advantages over centralized	**E-S-I-R-I**: Economics, Speed, Inherent distribution, Reliability, Incremental growth
8 transparencies	**A-L-M-R-R-C-F-P** ("All Lazy Men Really Relax, Chatting For Pleasure")
8 challenges	**H-O-S-S-F-C-T-Q**
3-tier benefits	**SR-FM-RS**: Scalability, Reliability, Flexibility, Maintainability, Reusability, Security
NIST characteristics	**O-B-R-R-M**
NIST models	**5-4-3**: characteristics, deployment, service
Service model audiences	**SaaS = Someone using; PaaS = Programmers; IaaS = Infrastructure admins**
Migration	**4 R's**: Rehost, Re-platform, Repurchase, Rebuild (cost rises)
Transformation terms	**E-C-S-M-H-E-S-L-V**
OLAP operations	**R-D-S-D-P**
Decision steps	**P-C-W-A-A-S-I-E**
Data quality	**A-V-R-T-R-C**
BI levels	Standard (What?), Ad hoc (How many?), Drill (Where?), Alerts (Action?)
BI layers	**S-W-P**: Source → Warehouse → Presentation

PART IV: MOCK EXAM AND ANSWERS

These questions are my own, based only on your notes. They are not the real exam. Try them with the answers covered.

Section A: Short answer / definitions (12 questions)

1. Define a distributed system and state the two aspects (hardware and software) the lecturer emphasises. *(Ch1)*
2. List any four types of transparency and say what each hides. *(Ch1)*
3. Name the three dimensions of scalability. *(Ch1)*
4. What is middleware, and where does it sit? *(Ch1)*
5. Distinguish a horizontal silo from a vertical silo. *(Ch2)*
6. State any four of the eight steps in integrating systems. *(Ch2)*
7. Name the three tiers of the ERP architecture and give two benefits of the three-tier approach. *(Ch3)*
8. What are WSDL, UDDI and SOAP? *(Ch3)*
9. Give the NIST definition of cloud computing and list its five essential characteristics. *(Ch4)*
10. Define CAPEX, OPEX and TCO. *(Ch4)*
11. What is an OLAP cube? Name its two components. *(Ch5)*
12. Define a data warehouse and state what “a decision” is. *(Ch5/Ch6)*

Section B: Explain / compare (6 questions)

13. Explain the goals of a distributed system and discuss why full transparency is not achievable. *(Ch1)*
14. Compare network operating systems, distributed operating systems and middleware. *(Ch1)*
15. Discuss the benefits and limitations of systems integration, and explain the role of ERP. *(Ch2)*
16. Explain SOA, including its contract, benefits and drawbacks. *(Ch3)*
17. Compare SaaS, PaaS and IaaS and the four cloud deployment models. *(Ch4)*
18. Compare OLTP and OLAP, and explain ROLAP, MOLAP and HOLAP. *(Ch5)*

Section C: Application / case study (3 questions)

19. **Case:** A Rwandan bank has an on-premise data center, strict regulations on customer data, and a yearly spike in transactions each December. (a) Recommend a cloud deployment model and justify it. (b) Explain cloud bursting for this bank. (c) Choose a

migration strategy for the bank's staff email system and justify it. *(Ch4)*

20. **Case:** A university's Finance, Admissions and Library systems cannot share data. (a) Identify the problem and explain logical vs physical integration. (b) Apply the 8 integration steps briefly. (c) State two limitations the university should expect. *(Ch2)*
21. **Case:** A supermarket chain wants to analyse sales. (a) Describe how data flows from its checkout systems to a manager's dashboard (ETL, warehouse, mart, OLAP). (b) Give one example each of roll-up, drill-down, slice and pivot. (c) Show how the manager would use the 8-step decision process to choose between two promotion plans. *(Ch5/Ch6)*

Section D: Calculations (2 questions)

22. An EC2 instance was available for 97.4% of a month. According to the SLA table, what credit is due? Show your working. *(Ch4)*
23. Two options are scored on cost (weight 0.6), risk (weight 0.4). Option X: cost 7, risk 5. Option Y: cost 5, risk 9. Which should be chosen? *(Ch6)*

ANSWER KEY

1. A DS is a collection of **independent computers that appear to users as a single coherent system**, coordinating by passing messages. **Hardware:** the computers are **autonomous**. **Software:** users **think they are dealing with a single system**.

2. Any four: **Access** (differences in data representation and invocation); **Location** (where an object resides); **Migration** (object can change location); **Relocation** (object moved while in use by a client); **Replication** (copies exist); **Concurrency** (coordination of shared use); **Failure** (failure and recovery); **Persistence** (disk/memory storage of the object).

3. Size (users/processes), **geographical** (distance between nodes), **administrative** (number of administrative domains).

4. The software layer **between the operating system and applications** that connects software components or enterprise applications across a network.

5. Horizontal silos divide by **function/department** (Accounting, HR; POSDCORB). **Vertical silos** divide by **management level** (strategic, tactical, operational).

6. Any four: resource categorization; compliance and standards; legacy systems support; middleware tools; authentication and authorization policies; centralized IT services and support; back-up, recovery and security; hardware and software standardization.

7. Web tier, application tier, data tier. Benefits (any two): scalability, reliability, flexibility, maintainability, reusability, security.

8. WSDL = Web Services Description Language; **UDDI** = Universal Description, Discovery and Integration; **SOAP** = Simple Object Access Protocol. They are **open, XML-based standards** for Web services.

9. *"A model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or service provider interaction."* Characteristics: **on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service.**

10. CAPEX = capital expenses on fixed assets (purchase, maintenance, improvement). **OPEX** = day-to-day operating costs. **TCO** = complete cost of an item/service over its whole lifetime, direct and indirect.

11. The OLAP cube (hypercube) is a data structure optimized for very fast analysis. Components: **measures** (numeric facts) categorized by **dimensions**.

12. Data warehouse: **subject-oriented, integrated, time-variant, non-volatile** collection of data supporting management decision making. A **decision** is making a **choice from two or more alternatives**.

13. *Marking points:* goals = **transparency** (8 types, hide distribution), **openness** (standard interfaces/IDL, portability, interoperability, policies vs mechanisms), **scalability** (size, geography, administration; techniques). Limits of transparency: **distribution is sometimes obvious** (different continents); **failures cannot be fully hidden** (slow vs failed computer; unknown whether an operation completed before a crash); **performance cost** (e.g., exact cache consistency, immediate disk flush).

14. *NOS:* each computer has its own OS; nodes independent, possibly different OSs; services tied to individual nodes; highly file-oriented. *DOS:* OS on each computer knows the others; services transparently distributed; hard for multicomputers (no shared memory, no simple global sync/broadcast); few exist (Amoeba). *Middleware:* layer above OSs; OSs need not know each other or be the same; transparent distribution; provides communication, information-system, control and security services; needed to integrate different NOSs/databases.

15. *Benefits:* increased revenue/growth; leveling competition; enhanced information visibility; increased standardization. *Limitations:* high set-up cost; power and interdepartmental conflicts; long-term intangible ROI; limits on creativity. *ERP:* integrated multi-module package that supports **logical integration** (focus on business processes, best practices, e.g., order entry → routing → communication) and **physical integration** (middleware, legacy replacement, data/client/application-level integration, EDI/B2B).

16. SOA breaks the business tier into **services**; any consumer on any OS/language can call a provider by messages. Contract: **functional description, input/output specifications, precondition, postcondition, error handling.** Benefits: heterogeneity, location independence, loose coupling, reliability, lower hardware cost, composite applications. Drawbacks: costly/time-consuming, complex firewalls and security, inconsistent performance, enterprise-level focus.

17. SaaS (apps for end users, e.g., Salesforce, Office 365), PaaS (platform for developers, API access, lock-in risk, e.g., Azure, Google App Engine), IaaS (infrastructure for sysadmins, e.g., EC2; base layer). Deployment: public (general public, cheapest, no location control), private (one organization, control, costly, limited scalability), hybrid (bridged, harder to manage, cloud bursting), community (shared concerns). Shared responsibility: duties shrink from IaaS to PaaS to SaaS; data always yours.

18. OLTP vs OLAP: use the 7-row table (user, function, design, data, view, usage, access). ROLAP: relational tables, scalable, slower. MOLAP: pre-computed multidimensional cube, fast, limited for un-stored aggregates. HOLAP: aggregates in cube, detail in relational DB, best of both.

19. (a) **Hybrid cloud**: keep regulated customer data in a **private** cloud/data center (control over location and security, legal compliance), use **public** cloud for less critical workloads. (b) **Cloud bursting:** run normally on private resources; when December demand exceeds local capacity, **burst to the public cloud**; avoids buying capacity that sits idle most of the year. (c) **Repurchase (to SaaS, e.g., Office 365)**: email is non-critical, frees IT from maintenance; (rehost is a faster alternative but gives no optimization).

20. (a) Problem: **silos**; **logical integration** = share data by need/authorization and change structures/processes/roles; **physical integration** = connect heterogeneous systems (middleware, standards). (b) Apply steps 1–8 (support, single sign-on/standards, legacy support, middleware, authentication, central help desk, back-up, standardization). (c) Any two: high set-up cost, power/departmental conflicts, long intangible ROI, reduced creativity.

21. (a) Checkout (**OLTP**) → **extract** → clean/transform → **load** into the **data warehouse** → **data mart** (e.g., Sales) → **OLAP cube** → **dashboard** (presentation layer). (b) Roll-up: city → country sales; drill-down: quarter → month; slice: time = Q1; pivot: swap product and region axes. (c) Steps: identify problem (sales below target); criteria (cost, risk, expected growth); weight them; list plans; analyse; choose highest score; implement and communicate; evaluate and repeat if needed.

22. Table: 99.0% to <99.5% → 10%; **95.0% to <99.0% → 30%**; below 95.0% → 100%. 97.4% lies in 95.0–<99.0 → **30% service credit.**

23. X = 0.6×7 + 0.4×5 = 4.2 + 2.0 = **6.2**. Y = 0.6×5 + 0.4×9 = 3.0 + 3.6 = **6.6**. **Choose Y** (higher total).

PART V: FINAL RAPID REVISION

5-minute sheet

CH1 – Distributed systems

- DS = independent computers, one coherent system, messages. **Goals:** Transparency (8: A-L-M-R-R-C-F-P), Openness (IDL, interfaces, mechanisms not policies), Scalability (size / geographic / administrative; techniques: hide latency, distribution, replication, caching → **inconsistency**).
- Adv: ESIRI + data/resource sharing, communication, flexibility. Disadv: software, network, security.
- 8 challenges; 8 false assumptions. Hardware: multiprocessor (shared memory) vs multicomputer (messages). Software: NOS (files) / DOS (few, Amoeba) / **middleware** (4 service groups).
- Architecture: entities, paradigm (IPC, RPC/RMI, indirect), roles (client–server, P2P), placement (caching, mobile code, mobile agents).

CH2 – Integration

- Silos: horizontal (function, POSDCORB) / vertical (level). Silo IS = inefficient, inaccurate, costly. BPR → cross-functional.
- Integration: **logical** (data sharing + org change) / **physical** (connect systems). **8 steps.** 4 benefits / 4 limitations. ERP = integrated multi-module package. Ethics: policy, security, training.

CH3 – ESA

- ESA = functional (modules) + system. Vendors SAP, Oracle/PeopleSoft, Microsoft Dynamics. **3-tier:** web / application / data (+4th web services). Server-centric = secure, slow; client-centric = fast, insecure. **SOA:** contract (function, I/O, pre, post, error); loose coupling; Web services: WSDL, UDDI, SOAP, XML. Cloud: lease, no licences, but vendor risk.

CH4 – Cloud

- NIST: **5** (self-service, broad access, pooling, elasticity, measured), **4** (public, private, hybrid, community), **3** (SaaS, PaaS, IaaS). Not a technology; a service model.
- Drivers: lower cost (CAPEX → OPEX) and agility. TCO. Migration: rehost (fastest) → re-platform → repurchase → rebuild (most expensive). SLA: EC2 99.5% → credits 10/30/100%. Multi-cloud: best-fit but complex. AWS: Regions → AZs (~3, ~100 km); Console / CLI / SDK. Shared responsibility: data is always yours.

CH5 – DW, OLAP, OLTP

- Architectures: single (virtual) / two (common) / three (reconciled). ETL terms: extracting, conditioning, scrubbing, merging, householding, enrichment, scoring, loading, validating.
- OLAP cube = measures × dimensions. **Roll-up, drill-down, slice (1 dim), dice (2+), pivot.** ROLAP (relational, slower), MOLAP (cube, fast), HOLAP (both); DOLAP, WOLAP, Mobile, SOLAP.
- OLTP: transactions, current, read/write, clerks. OLAP: decisions, historical, read, knowledge workers.
- Data mart: departmental. 12 pitfalls (cleaning time, high maintenance…).

CH6 – BI

- **8 steps:** problem, criteria, weights, alternatives, analyse, select, implement, evaluate. Levels: operational (structured), tactical (semi), strategic (unstructured).
- Cycle: data → information → knowledge → decision → action. Data quality: A-V-R-T-R-C.
- BI = turns data into information for decisions; = DSS. Components: **DW + data mining + business analyst.** Mining: classification, clustering, association, prediction; process pre-processing → mining → validation. Layers: source → warehouse → presentation. Questions: What happened / How many / Where / What action? 10 benefits.

Self-test (answers hidden below)

1. Which transparency hides that an object moved while a client was using it?

2. What problem does replication cause in scalability?
3. What is the difference between logical and physical integration?
4. Name two drawbacks of server-centric architecture and client-centric architecture (one each).
5. Which service model gives developers a platform and carries high lock-in risk?
6. What credit applies if EC2 uptime is 94%?
7. Which migration strategy is the fastest?
8. What does slicing do compared with dicing?
9. Which OLAP type stores aggregates in a cube and detail in a relational database?
10. Name the 8 steps of decision making in order.

Scroll past the line below for answers.

Self-test answers

1. **Relocation transparency.**
2. **Inconsistency** between copies; keeping them consistent needs global synchronization.
3. Logical: share data and change organization/processes/roles. Physical: connect heterogeneous systems technically.
4. Server-centric: **slower**. Client-centric: **lacks security.**
5. **PaaS.**
6. Below 95.0% → **100% credit.**
7. **Rehosting (lift-and-shift).**
8. Slice selects **one** dimension to make a sub-cube; dice selects **two or more.**
9. **HOLAP.**
10. Identify problem → identify criteria → allocate weights → develop alternatives → analyse alternatives → select alternative → implement → evaluate effectiveness.

FINAL COVERAGE CHECKLIST

Chapter	File	Covered?	Notes / unresolved gaps
1 Intro to Distributed Systems	Chapter_1.pptx	**Yes** (all text slides)	Image-only slides 28, 44, 48, 52, 56 (hardware diagram, RPC, multi-server, caching example, application layering) not readable; described from surrounding text. NOS/DOS/middleware comparison values are **not on the slides**; table is **[Added]**.
2 Systems Integration	Chapter_2.ppt	**Yes**	Slides 22–58 duplicate Chapter 3 (taught once in Ch3). Figures 2-1 to 2-6 not interpretable. Steps 2 and 5 have near-identical wording.
3 Enterprise Systems Architecture	Chapter_3.ppt	**Yes**	Figures 3-1 to 3-6 all show the same consumer/provider image after conversion; content taken from slide text. “Services junked often” wording unclear.
4 Cloud Computing	Chapter_4 pptx	**Yes** (all slides and notes)	Shared-responsibility colour split for PaaS runtime/middleware **[Added]**. Microservices/serverless detail referred to “Chapter 3”, not supplied.
5 Data Warehouse, OLAP, OLTP	Chapter_5 pptx	**Yes** (slides 1–70; 71–76 are references)	Roll-up example sentence reversed; number of OLAP operations inconsistent (3/4/5); scrubbing defined in hardware sense; some OLTP and HOLAP wording muddled. OLAP-vs-OLTP table taken from the Chapter 6 image.

Chapter	File	Covered?	Notes / unresolved gaps
6 Decision Making and BI	Chapter_6 pptx	**Mostly**	Slide 58 (“BI Types”) and picture slides 11, 22, 32, 34–36, 41–42, 44 not interpreted. Gartner capability grouping unclear.

Overall: all six files were opened and their text fully processed. I could **not** read every picture-only slide (listed above), so **please compare those few slides with your originals** before relying on them. Items marked **[Added]** are teaching aids, not lecturer content.